

Online Department Requisition & Store Management System

**A PROJECT REPORT
for
Major Project-1 (CA401P)
Session (2025-26)**

Submitted by

Udit Ranjan
(2400290140227)
Suchita Singh
(2400290140210)

**Submitted in partial fulfilment of the
Requirements for the Degree of**

MASTER OF COMPUTER APPLICATION

**Under the Supervision of
Dr. Neelam Rawat, Dr. Amit Kumar Gupta
(Asso. Professor) (Professor)**



Submitted to

**DEPARTMENT OF COMPUTER APPLICATIONS
KIET Group of Institutions, Ghaziabad
Uttar Pradesh-201206**

(MAY 2025)

CERTIFICATE

Certified that **Udit Ranjan 2400290140227, Suchita Singh 2400290140210** has/ have carried out the project work having “**Online Department Requisition & Store Management System**” (CA401P) for **Master of Computer Application** from KIET Group of Institutions (an Autonomous college) under Dr. A.P.J. Abdul Kalam Technical University (AKTU), Lucknow under my supervision. The project report embodies original work, and studies are carried out by the student himself/herself, and the contents of the project report do not form the basis for the award of any other degree to the candidate or to anybody else from this or any other University/Institution.

Date:

Udit Ranjan (2400290140227)

Suchita Singh (2400290140210)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Date:

**Dr. Neelam Rawat (Asso. Professor),
Dr. Amit Kumar Gupta (Professor),
Department of Computer Applications
KIET Group of Institutions, Ghaziabad**

**Dr. Sachin Malhotra
Dean (MCA)
Department of Computer Applications
KIET Group of Institutions, Ghaziabad**

Online Department Requisition & Store Management System

ABSTRACT

Managing supplies in colleges and universities has always been a tedious job. Staff members fill out paper forms, hand them to administrators, and then wait — sometimes for days — without any clear update on whether their request was approved or even received. This back-and-forth not only wastes time but also makes it nearly impossible to keep an accurate record of what is being used, what is running low, and who requested what. Existing software solutions do exist, but most of them are built for commercial warehouses and do not account for the way academic institutions actually function — with multiple departments, varying user roles, and semester-based inventory cycles.

This project was built to fix exactly that. The Requisition and Store Management System is a web-based platform that lets college staff raise supply requests online, track their status in real time, and receive updates directly in their inbox. The system is organized around three types of users — regular staff, administrators, and super users — each with a clearly defined set of permissions. An administrator can approve or reject requests, add comments, manage stock levels, and even handle special requests that fall outside the normal process. A super user can additionally manage who gets admin access, making the hierarchy clean and auditable.

On the technical side, the application is built using Flask and Python, with SQLite for development and MySQL for production environments. Email notifications are sent automatically whenever an account is created, a request is processed, or a password needs to be reset. The interface is clean and straightforward, built with Bootstrap and standard HTML templates so that even non-technical staff can navigate it without training.

Ultimately, what this system offers is something simple but valuable — clarity. Staff know where their requests stand, administrators have a complete picture of institutional supply consumption, and the institution as a whole gain a reliable tool that supports both day-to-day operations and longer-term quality assurance goals.

Keywords: Requisition Management, Supply Chain Automation, Role-Based Access Control, Inventory Tracking, Flask, Quota Enforcement, Automated Notifications, Web Application, Educational Administration, Institutional Accountability.

ACKNOWLEDGEMENT

Success in life is never attained single-handedly. My deepest gratitude goes to my project supervisor, **Dr. Neelam Rawat, Dr. Amit Kumar Gupta** for their guidance, help, and encouragement throughout my project work. Their enlightening ideas, comments, and suggestions.

Words are not enough to express my gratitude to **Dr. Sachin Malhotra**, Professor and Dean, Department of Computer Applications, for his insightful comments and administrative help on various occasions.

Fortunately, I have many understanding friends, who have helped me a lot on many critical conditions.

Finally, my sincere thanks go to my family members and all those who have directly and indirectly provided me with moral support and other kind of help. Without their support, completion of this work would not have been possible in time. They keep my life filled with enjoyment and happiness.

Suchita Singh

Udit Ranjan

TABLE OF CONTENTS

S. No.	Title	Page No.
	Certificate	ii
	Abstract	iii
	Acknowledgement	iv
	Table of Contents	v-vi
	List of Abbreviations	vii-viii
	List of Tables	ix
	List of Figures	x-xi
	CHAPTER 1: Introduction	1-9
1.1.	Background of the Problem	1-2
1.2.	Problem Statement	3-4
1.3	Aims and Objectives	4-6
1.4	Scope of Project	6-7
1.5	Hardware and Software requirement	7-9
	CHAPTER 2: Introduction to Existing System	10-15
2.1.	Existing Solutions and its limitation	10-14
2.2	Motivation	14-15
	CHAPTER 3: Proposed System	16-22
3.1.	Proposed Solution	16-17
3.2.	Key Features	17-20
3.3.	Advantages over Existing System	20-22
	CHAPTER 4: System Architecture	23-28
4.1	Overall System Design	23-24
4.2	Architecture Diagram	24-25
4.3	Modules Description	25-28
	CHAPTER 5: Technology Stack	29-34
5. 1	Programming Languages	29-31

5.2.	Frameworks & Libraries	31-32
5.3.	Database and Tools	32-34
	CHAPTER 6: System Design	35-53
6.1.	Use Case Diagram	35-41
6.2.	Data Flow Diagram (DFD)	42-43
6.3.	ER Diagram	44-49
6.4.	UML Diagrams	50-53
	CHAPTER 7: Implementation	54-64
7.1	Frontend Implementation	54-57
7.2	Backend Implementation	57-62
7.3	Database Design	63-64
	CHAPTER 8: Application Demonstration / Screenshots	65-77
8.1	User Interface Screens	65-76
8.2	Core Functionality Demonstration	77
	CHAPTER 9: Testing	78-82
9.1	Testing Strategies	78-79
9.2	Test Cases and Results	79-82
	CHAPTER 10: Results and Analysis	83-85
10.1	Performance Evaluation	83-84
10.2	Result Analysis	84-85
	CHAPTER 11: Conclusion	86-87
	CHAPTER 12: Future Scope	88-91
	CHAPTER 13: References	92-98

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
BCRYPT	Blowfish Crypt - Password Hashing Algorithm
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DB	Database
DFD	Data Flow Diagram
ER	Entity-Relationship
ERP	Enterprise Resource Planning
Flask	A lightweight Python web framework
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
ID	Identifier
IT	Information Technology
JS	JavaScript
JSON	JavaScript Object Notation
MIT	Massachusetts Institute of Technology (License)
MVC	Model-View-Controller
MySQL	My Structured Query Language
NAAC	National Assessment and Accreditation Council
NIRF	National Institutional Ranking Framework
ORM	Object-Relational Mapping
PEP	Python Enhancement Proposal

Abbreviation	Full Form
PIL	Python Imaging Library
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
SQLite	Self-Contained SQL Database Engine
TLS	Transport Layer Security
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator
UX	User Experience
WSGI	Web Server Gateway Interface
XSS	Cross-Site Scripting

LIST OF TABLES

Table No.	Table Title	Page No.
1	Hardware Requirements	7-8
2	Software Requirements	8-9
3	Comparison Sumary	13-14
4	Existing Systems	20-22
5	Frameworks and Libraries	31-32
6	Development and Deployment tools	34
7	Entity Description And Attributes	44-47
8	Route Handler	58-61
9	Form Definitions and Validators	61-62
10	Functional Test Cases	79-82
11	Response Time Analysis	83-84
12	Result Analysis	84-85

LIST OF FIGURES

Figure No.	Figure Title	Page No.
6.1	Use Case Diagram - Regular User	37
6.2	Use Case Diagram - Administrator	39
6.3	Use Case Diagram - Super User	41
6.4	DFD - Context Diagram	43
6.5	ER Diagram - Database Schema	49
6.6	Sequence Diagram - UserLogin	52
6.7	Sequence Diagram - Request Submission	53
8.1	Login Page Screenshot	66
8.2	Home Page - Regular User View	67
8.3	Home Page - Admin View	68
8.4	Category Page - I	69
8.5	Category Page - II	69
8.6	Stock Listing Page - I	70
8.7	Stock Listing Page - II	71
8.8	Request Form Page	72
8.9	User Request Summary	73
8.10	Admin Request List - I	74

Figure No.	Figure Title	Page No.
8.11	Admin Request List - II	74
8.9	Admin Stock List Page	75
8.13	User Profile Page	76

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND OF THE PROBLEM

Managing supplies in a college or university has always been more complicated than it looks from the outside. On the surface, it seems simple enough: a faculty member or lab staff needs some stationery or equipment, they ask for it, and someone provides it. But in practice, the process has never been that clean. Requests get lost. Administrators have no way of knowing which departments are over-consuming. Staff members submit the same request multiple times because they never heard back on the first one. And at the end of each semester, nobody can really say with confidence how much was used, how much is left, and how much needs to be ordered for the next cycle.

Educational institutions, particularly engineering colleges, deal with a wide variety of consumable items spread across multiple departments and laboratories. Stationery like pens, notebooks, markers, registers, files, and folders are needed regularly by both teaching and non-teaching staff. Lab equipment including chalk, lab books, printing paper, and cleaning materials require periodic replenishment. The challenge is not just in keeping these items available but in tracking exactly who is requesting what, whether those requests are justified, and how approvals are being handled.

Traditionally, all of this has been managed through paper. A staff member fills out a requisition form, hands it to their department head or directly to the administrative office. At each stage, there is no digital record of the request, no automatic notification, no way to check the status, and no way for the central office to see a consolidated picture of all ongoing requests across departments. If a request gets misplaced or forgotten at any point

in this chain, the staff member has no way of knowing until they physically follow up with someone.

This is not just an inconvenience. It represents a real inefficiency in how institutional resources are managed. Supplies get depleted faster in some departments while sitting unused in others. Budgets are harder to plan because historical consumption data is not tracked in any structured way. Accountability suffers because there is no audit trail to review when discrepancies arise. When accreditation bodies like NAAC or NIRF visit an institution and ask for evidence of resource management practices, the absence of documented systems becomes a significant concern.

The problem is made more complex by the fact that academic institutions operate on semester cycles. Inventory needs to be reviewed, reset, and replanned at the start of each semester. Without a system that supports this cycle, administrators end up doing the work manually every six months, which is both time-consuming and error-prone. Previous semester consumption data, which should inform future procurement decisions, is either unavailable or buried in filing cabinets.

The COVID-19 pandemic and the resulting shift to remote work further exposed these weaknesses. When staff members could not physically come to the institution to submit paper forms or follow up in person, the entire supply request process effectively stopped functioning. This highlighted the urgent need for a digital solution that could operate regardless of physical presence.

Against this backdrop, the need for a structured, digital, and transparent supply requisition system becomes clear. Such a system needs to be simple enough for all staff to use without technical training, comprehensive enough to handle the full lifecycle of a request, and robust enough to maintain accurate records across academic cycles.

1.2 PROBLEM STATEMENT

The core problem this project addresses is the absence of a structured, transparent, and trackable digital system for managing supply requisitions within an educational institution. The existing manual process suffers from several interconnected issues that compound over time and collectively undermine the efficiency, fairness, and accountability of institutional resource management.

Lack of Centralized Visibility

When a staff member submits a paper form, it enters a chain of handling that is invisible to everyone outside that chain. The person who submitted the request cannot track it. Other administrators do not know about it unless directly told. There is no single place where the status of all ongoing requests can be seen at once. This creates a situation where the same request may be duplicated, where urgent requests sit waiting while less important ones are processed first, and where administrators have no way to prioritize their workload intelligently.

No Enforcement of Fairness

Without a quota system, some users can request the same items repeatedly while others who need them equally urgently wait in line. Over-requisitioning is common because there is nothing stopping it. This leads to uneven distribution of resources, early depletion of budgets, and frustration among staff members who find their legitimate requests denied because stock has already been consumed by others.

Manual and Delayed Communication

If an administrator rejects a request or approves only a partial quantity, the requesting staff member has to be informed through a separate channel such as a phone call, a physical note, or an email composed manually. This extra step is frequently skipped or delayed, leading to confusion about whether a request is pending, approved, or rejected. Staff members who do not receive a response after a reasonable time often resubmit the request, creating duplicate entries that further complicate the administrator's workload.

Absence of Historical Records

At the end of each semester, whatever paper records exist are either filed away in cabinets or simply discarded. This makes it nearly impossible to analyze patterns in resource consumption, plan future procurement accurately, or produce reports for institutional audits. When accreditation bodies or internal review committees request evidence of resource management practices, the institution has no structured data to present.

No Support for Exceptional Cases

The manual system has no formal mechanism for handling exceptional requests. When a staff member has a genuine need for more supplies than their normal allocation would suggest, there is no official pathway to escalate the request for higher-level review. These exceptions are handled informally and inconsistently, leading to both under-supply for legitimate needs and potential misuse.

The problem statement, in its simplest form, is this: how do you take a process that is currently invisible, inconsistent, and manual, and replace it with something that is transparent, fair, and automated, without requiring major technical expertise from the end users who will interact with it daily?

1.3 AIMS AND OBJECTIVES

The primary aim of this project is to develop a web-based application that digitizes and streamlines the complete requisition lifecycle within an educational institution, from the initial request submission by a staff member to the final delivery confirmation and historical reporting. The system should serve as a single source of truth for all supply-related activities and provide appropriate interfaces for each level of the institutional hierarchy.

Specific Objectives

- To design and implement a clean, browser-based web interface that allows staff members to browse available inventory items by category or as a complete list, select what they need, specify a quantity, and submit a formal request with a supporting justification message, all without requiring any technical training or installation on their device.
- To implement a structured multi-step approval workflow where administrators can view all incoming requests in a consolidated queue, approve them with a possibly modified quantity, reject them with a reason, and add comments that are immediately visible to the requesting user upon their next login.
- To enforce per-user, per-item quota limits for each semester automatically within the system, so that resource distribution remains fair and no single user can over-request items beyond their allocated share, with the quota remaining enforced even if a user submits multiple smaller requests that together would exceed the limit.
- To provide a dedicated special request pathway for situations where a user has exhausted their quota but has a genuine need for additional supplies, with this secondary track being handled exclusively by a higher-authority super user through a separate review queue.
- To maintain a complete and searchable log of all requests, approvals, rejections, and delivery confirmations with timestamps and responsible party information, so that historical data is always available for review, reporting, and institutional auditing.
- To automate email communication at key events including new account creation with credentials, password reset with a time-limited secure link, and account deletion notification, so that all stakeholders are kept informed without requiring manual communication effort.

- To support semester-wise inventory management including one-click stock resets with automatic archival of previous semester data and CSV export, so that administrators can plan procurement for each new academic cycle based on actual historical consumption.
- To implement role-based access control with three distinct levels of authority ensuring that all operations are accessible only to users with appropriate permissions, and that these checks are enforced at the server level rather than just hidden in the user interface.
- To ensure the system is responsive and accessible on multiple device types including desktops, laptops, tablets, and smartphones without requiring separate development for each platform.

1.4 SCOPE OF PROJECT

The scope of this project covers the complete cycle of supply requisition management within a single institutional deployment. The system is intended to serve all staff of a college including lab assistants, teaching staff, and non-teaching staff on the user side, and department administrators and institutional super users on the management side.

Included in Scope

- Complete user authentication including registration, login, logout, and password reset via email
- Three-tier role management covering regular users, administrators, and super users
- Inventory management covering categories and stock items with all associated parameters
- Standard request submission, approval, rejection, and delivery confirmation workflow
- Special request submission and super-user review workflow

- Quota calculation and enforcement per user per item per semester
- Email notifications for account creation, password reset, and account deletion
- CSV export of stock data for reporting and record-keeping
- Semester-wise stock reset with archival of previous data
- Real-time client-side search across all major data tables
- Profile management including name, email, and profile photo update
- Responsive web interface accessible from any device with a browser

Excluded from Scope

- Procurement from external vendors or vendor management functionality
- Financial billing, cost tracking, or budget management (planned for future version)
- Integration with external ERP systems or HR management software
- Multi-institution or multi-tenant deployment in this version
- Native mobile application development for iOS or Android
- Barcode or QR code scanning for physical inventory management
- Automated low-stock alerts or purchase order generation

1.5 HARDWARE AND SOFTWARE REQUIREMENTS

1.5.1 Hardware Requirements

TABLE I
HARDWARE REQUIREMENTS

Component	Minimum Specification	Recommended Specification	Notes
Processor	Intel Core i3 or equivalent	Intel Core i5 or equivalent	Any modern processor sufficient

Component	Minimum Specification	Recommended Specification	Notes
RAM	2 GB	4 GB or more	More RAM improves development experience
Storage	10 GB free disk space	20 GB or more	For OS, Python, database, and media files
Network	Standard LAN connection	Broadband internet connection	Required for email functionality
Display	1024 x 768 resolution	1920 x 1080 resolution	Bootstrap UI is responsive
Web Server	Any WSGI-compatible server	Gunicorn on Linux	As specified in Procfile

1.5.2 Software Requirements

TABLE II
SOFTWARE REQUIREMENTS

Component	Technology / Version	Purpose
Operating System	Ubuntu 18.04+ / Windows 10	Server or development environment
Python	3.6 or higher	Primary backend programming language
Flask	Latest stable	Web framework for routing and request handling

Component	Technology / Version	Purpose
Flask-SQLAlchemy	Latest stable	ORM for database abstraction
Flask-Login	Latest stable	User session and authentication management
Flask-Bcrypt	Latest stable	Secure password hashing
Flask-Mail	Latest stable	Email sending via SMTP
Flask-WTF / WTForms	Latest stable	Form handling and CSRF protection
itsdangerous	Latest stable	Cryptographic token generation for password reset
Pillow	Latest stable	Image processing and resizing
SQLite 3	Built-in with Python	Development database
MySQL 5.7+	5.7 or higher	Production database
Bootstrap	4.0.0	Responsive frontend CSS framework
jQuery	3.2.1	DOM manipulation and event handling
Font Awesome	5.0.7	Icon library
Git	Latest	Version control
Heroku CLI	Latest	Cloud deployment
Web Browser	Chrome / Firefox / Edge	Client-side interface

CHAPTER 2

INTRODUCTION TO EXISTING SYSTEM

2.1 EXISTING SOLUTIONS AND THEIR LIMITATIONS

Before designing any new system, it is essential to thoroughly understand what currently exists and why it falls short of what is needed. Supply and inventory management is not a new problem, and organizations of all sizes have developed various approaches to handle it. For educational institutions specifically, the available options range from completely manual paper-based processes to complex enterprise software designed for commercial operations. Each of these has its own set of strengths, but each also comes with limitations that make it unsuitable as a complete solution for the academic institution context.

2.1.1 Paper-Based Manual Systems

The most common approach in small and medium-sized educational institutions, particularly in India, is still the paper requisition form. A staff member fills in a form that typically includes their name, department, date, a list of required items with quantities, and a signature. This form is then routed through a chain of approval: first to the department head for initial endorsement, then to the administrative office for processing, and finally to the storeroom for fulfillment.

On paper, this seems functional. In practice, it creates multiple compounding problems. The form can be lost at any point in the chain, and there is no way for the requester to know this has happened until they physically follow up. Processing time depends entirely on the availability of the individuals in the chain, meaning a busy department head can inadvertently delay dozens of requests simply by not getting to their

pile of forms. There is no consistent format for recording the outcome of each request, so different administrators handle rejections and partial approvals in different ways.

From an institutional perspective, paper forms generate no useful data. At the end of each semester, the institution has a pile of processed forms but no easy way to answer basic questions like which items are most frequently requested, which departments consume the most resources, or how consumption this semester compares to last semester. This absence of structured data makes procurement planning essentially guesswork.

2.1.2 Spreadsheet-Based Tracking

Some institutions have moved beyond pure paper to using shared spreadsheets, typically Microsoft Excel or Google Sheets. This represents an improvement in that data is at least captured digitally and can be searched, filtered, and aggregated. However, spreadsheets introduce their own category of problems that make them unsuitable as a long-term solution.

Access control is the most significant issue. A shared spreadsheet has no concept of roles or permissions. Anyone with access can accidentally or intentionally modify, delete, or corrupt data. There is no approval workflow built into a spreadsheet; the workflow still happens through other channels, and the spreadsheet is updated manually after the fact. This means the spreadsheet is always a record of what has been decided rather than a tool for making decisions.

Scalability is another problem. A spreadsheet that starts as a simple tracking tool quickly becomes unwieldy as the number of users, items, and requests grows. Formula errors accumulate. Conditional formatting rules become complex. Multiple people trying to edit simultaneously create conflicts. Maintaining consistency in how data is entered requires discipline that is difficult to enforce across a large group of users.

Version history, while available in cloud-based spreadsheets, is not a substitute for proper audit logging. It shows what changed and when but not why, and navigating it to investigate discrepancies is cumbersome.

2.1.3 Commercial ERP Systems

At the other end of the spectrum, enterprise resource planning systems such as SAP, Oracle ERP, and Tally offer comprehensive modules for inventory management, procurement, and supply chain operations. These systems are powerful, feature-rich, and battle-tested in large commercial environments.

However, they are fundamentally mismatched for the educational institution use case in several important ways. The cost of licensing, implementation, and ongoing maintenance is typically prohibitive for academic institutions with limited IT budgets. A single SAP implementation can cost several times what a typical engineering college spends on IT in an entire year.

The complexity of these systems requires dedicated IT staff with specialized training to operate and maintain them. Most colleges cannot support this. The user interfaces, designed for commercial procurement workflows involving vendors, purchase orders, and financial ledgers, are far more complex than what is needed for simple internal requisition management. The learning curve for non-technical staff is steep, and resistance to adoption is high.

Customization to fit the specific needs of academic operations, such as semester-based inventory cycles or the three-tier academic hierarchy, typically requires expensive consulting engagement and ongoing maintenance as the software vendor releases updates.

2.1.4 Generic Web-Based Inventory Tools

There are various open-source and commercial web-based inventory management tools available, such as Snipe-IT, InvenTree, and similar platforms. These tools are more modern in their approach and more accessible than enterprise ERP systems, but they are still designed with different use cases in mind.

Most of these tools are designed for asset management, meaning they track specific physical items with serial numbers rather than consumable stock quantities. They lack the concept of a user-facing request form that routes through an approval workflow. They do

not support per-user quotas. They do not have the concept of a semester-based reset cycle. Adapting them to the educational institution requisition context would require significant custom development, at which point it would have been simpler to build a purpose-built solution from the start.

2.1.5 Comparison Summary

TABLE III
COMPARISON SUMMARY

Feature / Criterion	Paper Forms	Spreadsheets	Commercial ERP	Generic Web Tools	Proposed System
Request Tracking	None	Manual	Yes	Partial	Full Real-time
Role-based Access	None	None	Yes	Partial	Three-tier
Quota Enforcement	None	None	Configurable	None	Built-in
Approval Workflow	Manual	Manual	Yes	None	Structured
Email Notifications	None	None	Yes	Partial	Automated
Semester Reset	Manual	Manual	Configurable	None	One-click
CSV Export	None	Manual	Yes	Yes	Built-in
Special Requests	Informal	None	Complex	None	Dedicated Track

Feature / Criterion	Paper Forms	Spreadsheets	Commercial ERP	Generic Web Tools	Proposed System
Setup Cost	None	Low	Very High	Medium	Low
Technical Complexity	None	Low	Very High	Medium	Low
Audit Trail	None	None	Yes	Partial	Complete

2.2 MOTIVATION

The motivation behind this project grew directly out of observing these gaps firsthand. The development team had direct experience with how the manual requisition process works in an academic institution and could see clearly where and why it was failing. Lab staff who submitted requests would wait for weeks without hearing anything. Administrators who received dozens of paper slips had no organized way to process them systematically. Department heads who needed to sign off on requests had no visibility into what was already approved or rejected for their department.

Several specific observations drove the decision to build this system rather than adapt an existing one. First, the users are not technical. Any solution needs to work with a standard web browser and require no installation, configuration, or specialized training. Second, the problem is specific. The general-purpose tools available solve different problems and would require substantial customization. Third, the institution is already digitizing in other areas and needs this process to catch up. And fourth, there is real institutional value in having documented, structured data about supply consumption, both for internal planning and for external accreditation purposes.

The decision to build the system using Python and Flask was motivated by several practical considerations. Python has a gentle learning curve and is already familiar to

many computer engineering students and faculty. Flask is lightweight and does not impose unnecessary structure. The combination of Flask-SQLAlchemy and SQLite means the system can be set up and running in minutes on any computer without database server installation, making development and testing straightforward. And the open-source MIT license under which this project is released reflects a belief that this kind of institutional tool should be freely available to any college that could benefit from it.

The motivation is ultimately both practical and principled. Practically, a better system saves time, reduces errors, and improves communication. Principally, institutions that serve students and staff deserve administrative processes that respect people's time and treat resource allocation fairly. This project is an attempt to bring both of those things to the supply requisition process.

CHAPTER 3

PROPOSED SYSTEM

3.1 PROPOSED SOLUTION

The proposed solution is a web-based Requisition and Supply Management System built using Python and the Flask micro-framework. The system is designed to replace the manual, paper-based process with a structured digital workflow that is accessible through any standard web browser, requires no installation on the user's device, and works consistently across desktops, tablets, and mobile phones through Bootstrap's responsive grid system.

At its core, the system creates a shared digital space where staff members can submit supply requests and administrators can process them, all within a single platform that maintains a permanent and searchable record of every action taken. The system is organized around the real-world roles that exist in a college: regular staff who need supplies, department administrators who approve those requests, and a super user at the top of the hierarchy who manages the system and handles exceptional cases.

The design philosophy throughout has been simplicity without sacrifice. The system does not try to solve every possible problem in supply chain management. It solves the specific problem of internal supply requisition in an educational institution, and it does so in a way that is immediately understandable to a first-time user. The navigation is organized by what users need to do rather than by how the underlying data is structured. The forms are short and ask only for information that is genuinely needed. The feedback at every step is clear and immediate.

The technical architecture follows the Model-View-Controller pattern. Data structures are defined as Python classes using SQLAlchemy, forming the Model layer. HTML templates using Jinja2 form the View layer. Flask route handlers in routes.py form the Controller layer. This separation makes the codebase maintainable and allows each layer to be modified independently.

3.2 KEY FEATURES

3.2.1 Request Enforcement

Every item in the system carries a value that represents the maximum total quantity any user can request for that item. The enforcement logic runs on the server side and cannot be bypassed from the browser. When a user submits a request, the system calculates the all approved and pending requests for that item, adds the new requested quantity, and compares the total.

This calculation correctly ignores rejected requests, so a user who had a request rejected is not penalized for it in their remaining quota. It also accounts for partial approvals: if a user requested ten units but was only approved for six, only six units are counted against their quota, and they retain the ability to request the remaining four.

The quota for each item is also displayed to the user before they submit a request, showing exactly how much of their allocation remains. This transparency prevents wasted effort in submitting requests that would be rejected.

3.2.2 Two-Track Requisition System

The system implements two separate request pathways that handle different situations. The standard request track is for normal requisitions where the user is within their quota. These requests appear in the main administrator request queue and follow the standard approval workflow.

The special request track is available exclusively when a user's remaining quota for a specific item is exactly zero. This prevents special requests from being used as a workaround for the quota system. When a user's quota is exhausted, the regular request option for that item is replaced by a special request option. Special requests are visible only to super users, not to regular administrators, ensuring they receive appropriate scrutiny from the highest level of authority. When a super user approves a special request, the system automatically creates a corresponding regular request record so the item appears in the user's standard request summary.

3.2.3 Three-Tier Role Hierarchy

The permission model is built around three roles that map to real organizational positions within a college. Regular Users, who are typically lab staff and teaching staff, can browse inventory, submit requests, track their request history, confirm delivery of approved items, and manage their personal profiles. Administrators, who are typically department heads or administrative officers, have all regular user capabilities plus the ability to view and process all requests from all users, manage the inventory catalog, manage user accounts, view consolidated summaries, and perform semester resets. Super Users have all administrator capabilities plus exclusive access to special request management and the ability to change other users' role assignments including granting or revoking administrator status.

These role checks are enforced at the route handler level in `routes.py`. Every admin route begins with a check that aborts the request with a 403 Forbidden response if the current user does not have the required role. This server-side enforcement is essential because it cannot be bypassed by manipulating the browser, unlike client-side role checks that only hide interface elements.

3.2.4 Automated Email Communication

The system automates three categories of email communication that would otherwise require manual effort from administrators. When a new user account is created, the system

automatically generates a secure random password, stores its hashed version in the database, and immediately sends the plain-text password to the new user's email address along with instructions to change it after first login. When a user requests a password reset, the system generates a cryptographically signed token that encodes the user's ID with a thirty-minute expiry window, and sends a link containing this token to the user's registered email. When a user account is deleted by an administrator or super user, the deleted user receives an email informing them of the deletion and identifying which administrator performed the action, so they know who to contact if the deletion was in error.

3.2.5 Semester-Wise Stock Management

The system is explicitly designed to support the academic semester cycle. The Stock model contains four quantity fields that together enable semester-wise tracking: `qty_prev` stores the consumption total from the previous semester, `avail` stores the current available quantity in the storeroom, `qty_req` stores the quantity that should be ordered for procurement planning, and `qty_pres` tracks the total consumed in the current semester through all approved requests.

At the end of each semester, an administrator can perform a stock reset by clicking a single button. This operation first generates and downloads a CSV file containing the current values of all four quantity fields for every stock item, creating a permanent downloadable record of the semester. It then copies the `qty_pres` value to `qty_prev`, archives the semester data, and resets `qty_pres` to zero for all items. The system is now ready for a new semester, with the previous semester's consumption data permanently recorded in the `qty_prev` field for reference.

3.2.6 Real-Time Client-Side Search

All major data tables in the system support real-time search filtering implemented in vanilla JavaScript. The search works by listening to keyup events on a search input field above the table, converting the entered text to uppercase for case-insensitive comparison,

and iterating through all table rows to show or hide them based on whether the target column of each row contains the search string. No server request is made during searching, so the filtering is instantaneous regardless of table size. Different tables search different columns: the stock table searches by item name, the request tables search by item name or request ID, and the summary table searches by item name.

3.2.7 Delivery Confirmation Loop

The request lifecycle includes a final step that closes the loop between approval and physical receipt. When an administrator approves a request, the user's request summary page shows a Received button next to that request. This button is only active for approved requests that have not yet been confirmed as received. When the user clicks Received after collecting the physical items, they can add a brief confirmation note and submit. This marks the request as accepted in the database, disables the Received button to prevent duplicate confirmations, and stores the delivery note for the administrator's records. This step is optional in the sense that the system continues to function if it is skipped, but it provides valuable confirmation that approved supplies were actually delivered rather than lost in transit.

3.3 Advantages Over Existing System

TABLE IV
EXISTING SYSTEMS

Aspect	Old Manual System	Proposed Digital System
Request Submission	Physical paper form, office visit required	Online form from any device, any location
Request Tracking	Impossible after form is submitted	Real-time status with green/red/clock icon

Aspect	Old Manual System	Proposed Digital System
Quota Enforcement	Relies entirely on administrator memory	Automatic server-side calculation and enforcement
Approval Time	Days to weeks depending on availability	Immediate on administrator action
Communication	Separate manual phone call or note	In-system status and comment, visible immediately
Special Cases	Handled informally, inconsistently	Dedicated special request track with super user review
Delivery Confirmation	None	In-system acknowledgment with optional note
Historical Records	Paper files, easily lost or damaged	Permanent database records, searchable, timestamped
Semester Planning	Manual review of paper records	One-click CSV export with complete historical data
User Accounts	Not applicable	Self-service password reset, profile management
Audit Capability	None	Complete trail: who requested, who approved, when, comment
Fairness	No enforcement, first-come first-served	Per-user quota ensures equitable distribution
Accessibility	Requires physical presence at institution	Web browser sufficient, works on any device
Scalability	Degrades with more users and items	Database-backed, handles growth naturally

Aspect	Old Manual System	Proposed Digital System
Reporting	Manual compilation, time-consuming	Automated CSV, instant download

CHAPTER 4

SYSTEM ARCHITECTURE

4.1 OVERALL SYSTEM DESIGN

The system is built on a classic three-tier web application architecture that separates the user interface, business logic, and data storage into distinct layers. This separation is not just an organizational choice; it has practical benefits for maintainability, testability, and future extensibility. A developer who wants to change the database backend needs to touch only the data layer. A designer who wants to redesign the interface needs to work only in the templates. A developer adding a new business rule needs to modify only the route handler.

Presentation Tier

The presentation tier is everything the user sees and interacts with. It consists of HTML templates written using the Jinja2 templating engine, styled with Bootstrap 4 and custom CSS files, and enhanced with vanilla JavaScript and jQuery for interactive elements. The templates are rendered on the server side by Flask and sent to the browser as complete HTML pages. This server-side rendering approach means the application works correctly even with JavaScript disabled for core functionality, while JavaScript adds progressive enhancement for search and inline editing.

Bootstrap 4's grid system provides the responsive layout that makes the interface work correctly on devices ranging from smartphones to large desktop monitors. The navigation bar collapses to a hamburger menu on small screens. Tables become horizontally scrollable on narrow viewports. The card-based home page layout reflows from a multi-column grid to a single column on mobile devices.

Application Tier

The application tier is the Flask application running on the Python web server. This tier contains all the business logic: route handlers that process incoming HTTP requests, form validation through Flask-WTF, authentication and session management through Flask-Login, password hashing through Flask-Bcrypt, email sending through Flask-Mail, image processing through Pillow, and token generation through itsdangerous. This is where decisions are made: whether a request is within quota, whether a user has permission to access a route, whether a form is valid before processing it.

The route handlers in `routes.py` are organized into three sections separated by comments: Admin routes prefixed with `/admin/`, User routes prefixed with `/user/` or `/make/`, and General routes that are accessible to all authenticated users. This organizational structure makes it easy to understand the permission model at a glance.

Data Tier

The data tier is managed by SQLAlchemy, which acts as an Object-Relational Mapper between the Python application and the underlying relational database. The application defines Python classes for each entity in the domain: User, Category, Stock, Request, and SpecialRequest. SQLAlchemy translates operations on these Python objects into SQL statements executed against the database. The database connection string in the configuration determines whether the application talks to SQLite during development or MySQL in production, and this is the only change needed to switch between them.

4.2 ARCHITECTURE DIAGRAM DESCRIPTION

The complete data flow through the system can be described as follows:

1. The user opens a web browser and navigates to the application URL.
2. The browser sends an HTTP GET or POST request to the Flask application server.
3. The Flask routing engine matches the URL to the appropriate route handler function in `routes.py`.

4. The route handler checks authentication status using Flask-Login. Unauthenticated requests are redirected to the login page.
5. The route handler checks the current user's role using the `isAdmin` and `isSuperUser` flags. Unauthorized requests receive a 403 response.
6. The route handler processes the request: validates form data using Flask-WTF, applies business logic such as quota calculation, performs database operations through SQLAlchemy models, and sends emails through Flask-Mail if required.
7. The route handler renders a Jinja2 template with the relevant data and returns the rendered HTML to the browser.
8. The browser renders the HTML, applies CSS styles, and executes JavaScript for interactive elements.
9. The user interacts with the interface, triggering the next HTTP request, and the cycle repeats.

4.3 MODULES DESCRIPTION

4.3.1 Authentication and Session Module

This module is responsible for controlling who can access the application and maintaining session state across requests. It is implemented using Flask-Login, which provides the `login_required` decorator that protects all routes requiring authentication. The `user_loader` function registered with the login manager retrieves the User object from the database using the user ID stored in the session cookie. Password verification uses bcrypt's `check_password_hash` function, which is resistant to timing attacks that could be used to probe valid credentials.

The password reset sub-module uses `TimedJSONWebSignatureSerializer` from the `itsdangerous` library. This serializer creates signed tokens that contain a payload (the user's ID) and an expiry time (thirty minutes). The signature is created using the application's secret key, so tokens created by one deployment cannot be used with a different deployment even if the token format is known. The `get_reset_token` method on

the User model generates the token, and the static `verify_reset_token` method decodes it and returns the corresponding User object, or None if the token is invalid or expired.

4.3.2 User Management Module

The user management module handles the full lifecycle of user accounts. Account creation is performed by administrators through a registration form that collects first name, last name, and email address. The system generates a random sixteen-character hexadecimal password using Python's `secrets.token_hex(8)` function, hashes it with `bcrypt`, stores the hash, and sends the plain-text password to the new user's email. This approach ensures that the system never needs to store or transmit plain-text passwords after the initial creation event.

Profile updates are handled through a dedicated form that allows users to change their name, email address, and profile photo. The form requires the user's current password for verification before any changes are saved, preventing unauthorized profile modification if a session is left open. Password changes have their own dedicated form that requires both the current password and a new password with confirmation.

Role management is restricted to super users. Toggle routes allow super users to flip the `isAdmin` and `isSuperUser` flags for any user account. The toggle logic is designed so that making a user a super user automatically also makes them an administrator, since super user is a superset of administrator privileges. Removing super user status does not automatically remove admin status, allowing for fine-grained role assignment.

4.3.3 Inventory Management Module

The inventory module manages two levels of the catalog hierarchy: categories and stock items. Categories are visual groupings with a name and a representative photograph. Each category can contain multiple stock items. Stock items carry all the quantitative information: available quantity (`avail`), required quantity (`qty_req`), previous semester consumption (`qty_prev`), current semester consumption (`qty_pres`), minimum threshold (`minimum_limit`), maximum threshold (`maximum_limit`), and per-user quota (`quota`).

The module provides administrators with full CRUD operations on both categories and items. Images for categories are uploaded through a file field, processed by the `save_picture` utility function which resizes them to 250 by 250 pixels using Pillow's `thumbnail` method, and stored in the `static/images/category/` directory with a random hexadecimal filename. The filename stored in the database points to this processed version.

The download route generates a CSV file on the fly using Python's `csv` module, writing header columns and one row per stock item. The reset route performs the semester transition: it generates and serves the CSV file, then updates all stock records to copy `qty_pres` to `qty_prev` and reset `qty_pres` to zero. The entire reset operation happens within a single database session to ensure consistency.

4.3.4 Request Processing Module

This is the central workflow module. The `make_request` route handler implements the core quota logic. When a POST request arrives with a stock ID, quantity, and message, the handler queries all existing requests for the current user and that stock item, filters for those with status 0 (pending) or 1 (approved), sums their quantities, and calculates `quota_left` as `max(0, stock.quota - total_used)`. If the new requested quantity exceeds `quota_left`, a flash error message is displayed and the form is re-rendered. If within quota, a new Request object is created with status 0 and committed to the database.

The `accept_request` handler validates the administrator's input before applying the approval. It checks that the approved quantity is numeric, that it does not exceed the originally requested quantity, and that it does not exceed the current available stock. If any check fails, an error message is displayed and no change is made. If all checks pass, the request's `qty` is updated to the approved amount, status is set to 1, `admins_comment` and `processed_by` are recorded, and the changes are committed.

The `reject_request` handler is simpler: it sets status to -1, records the admin's comment and name, and commits. The `request_received` handler validates that the current user owns the request before setting `accepted` to True and saving the `received_comment`.

4.3.5 Special Request Module

The special request module parallels the standard request module but operates through a separate database table and a separate approval queue visible only to super users. The `make_special_request` handler includes an additional validation step: it calculates the user's remaining quota for the requested item, and if any quota remains, it rejects the special request with an error message explaining that special requests are only for quota-exhausted situations. This prevents the special request pathway from being used as a way to request items while still having regular quota available.

When a super user approves a special request, the handler does two things: it updates the `SpecialRequest` record to status 1, and it creates a new `Request` record with the same details. This duplication is intentional: it ensures that the item appears in the user's standard request summary, where they will see its status and can confirm receipt, while the `SpecialRequest` record remains in the special requests system as a record of the exception.

4.3.6 Email Notification Module

The email module uses Flask-Mail configured against Gmail's SMTP server on port 587 with TLS encryption. Three email functions are defined as utility functions at the bottom of `routes.py`. The `send_reset_email` function generates a reset token and constructs an email with the reset link using Flask's `url_for` with the `_external=True` parameter to generate an absolute URL including the domain name. The `send_create_user_email` function constructs a welcome email with the new user's credentials. The `send_delete_account_email` function constructs a deletion notification. All three functions use Flask-Mail's `Message` class and the `mail.send()` method.

CHAPTER 5

TECHNOLOGY STACK

5.1 PROGRAMMING LANGUAGES

5.1.1 Python 3.6+

Python is the backbone of the entire server-side application. It was chosen for this project for several overlapping reasons. Its syntax is clean and readable, which makes the codebase easier to maintain and review. Its ecosystem of web development libraries is mature and well-documented. It is already familiar to computer engineering students from their coursework. And it runs without modification on Windows, Linux, and macOS, making development convenient regardless of the team member's preferred operating system.

The project follows PEP8 coding standards, which are explicitly referenced in the README. PEP8 defines guidelines for code formatting including indentation, line length, naming conventions, and import organization. Following these standards makes the code more consistent and easier for other Python developers to read and contribute to.

Python 3.6 specifically is the minimum required version because of features used in the codebase including f-strings for string formatting (though the code uses older-style `.format()` in some places), the `secrets` module for cryptographically secure random generation, and type hints in some places. Modern Python 3.8 or 3.9 is recommended for deployment.

5.1.2 HTML5 and CSS3

HTML5 provides the markup structure for all of the application's pages. The use of semantic HTML5 elements such as `header`, `nav`, `main`, `footer`, and `section` throughout the

templates improves accessibility and search engine discoverability. Form elements use HTML5 type attributes like email, number, and password which trigger appropriate keyboard types on mobile devices and provide basic client-side validation through the required attribute and min/max constraints.

CSS3 is used for all custom styling beyond Bootstrap's defaults. Separate CSS files are maintained for different sections of the interface: login.css handles the full-page centered login form, navbar.css styles the navigation bar, request.css styles the request form and table, search.css styles the search input and results, user.css handles user-specific elements, and editProfile.css handles the profile editing interface. This separation prevents CSS rules from unintentionally affecting unrelated parts of the interface.

The color scheme throughout the application uses dark red (#9d0b0b) as the primary brand color, appearing on buttons, card headers, and the login page background. This color is defined as a CSS custom class called .bgcolor and .colors and applied consistently. The navigation bar uses a light grey (#e8f1f5) background to keep the header visually distinct from the content area.

5.1.3 JavaScript (ES6)

JavaScript is used for client-side interactivity. The application uses vanilla JavaScript for the search and table editing functionality and jQuery for DOM manipulation and animations. The use of vanilla JavaScript for the search functionality means there is no dependency on any JavaScript library for this feature, reducing the risk of compatibility issues.

The search.js file contains four separate search functions for different tables. Each function follows the same pattern: get the input element by ID, read its current value, convert to uppercase, find the target table by ID, iterate through all tr elements, find the target td by index, read its text content, and compare. Rows that do not match are hidden by setting display to 'none', and rows that do match have their display restored.

The `stocks.js` file contains the `edit_row` function that converts static table cells into editable input fields for inline stock quantity editing. The `user.js` file contains the `editRow` function for toggling the received confirmation form, and a jQuery `setTimeout` call that automatically fades out alert messages after three seconds.

5.2 FRAMEWORKS AND LIBRARIES

TABLE V
FRAMEWORKS AND LIBRARIES

Library	Version	Purpose	Key Features Used
Flask	2.x	Core web framework	Routing, request handling, template rendering, flash messages
Flask-SQLAlchemy	Latest	ORM integration	Model definitions, session management, query API
Flask-Login	Latest	Authentication	<code>login_required</code> decorator, <code>current_user</code> proxy, <code>user_loader</code>
Flask-Bcrypt	Latest	Password security	<code>generate_password_hash</code> , <code>check_password_hash</code>
Flask-Mail	Latest	Email delivery	Message class, <code>mail.send()</code> , SMTP configuration
Flask-WTF	Latest	Form handling	FlaskForm base class, CSRF token injection
WTForms	Latest	Form fields	StringField, PasswordField, IntegerField, FileField, validators
itsdangerous	Latest	Token generation	TimedJSONWebSignatureSerializer for password reset tokens

Library	Version	Purpose	Key Features Used
Pillow	Latest	Image processing	Image.open(), thumbnail(), save() for profile and category photos
Jinja2	Latest	HTML templating	Template inheritance, conditionals, loops, filters
Bootstrap	4.0.0	CSS framework	Grid system, cards, modals, navbar, responsive utilities
jQuery	3.2.1	DOM library	fadeOut animation, click handlers, DOM traversal
Font Awesome	5.0.7	Icon library	fa-check-circle, fa-times, fa-clock, fa-plus-circle icons
Popper.js	1.12.9	Bootstrap dependency	Dropdown and tooltip positioning

5.3 DATABASE AND TOOLS

5.3.1 SQLite

SQLite is used as the development database. It is a serverless, self-contained relational database engine that stores the entire database in a single file. The SQLite database file for this project is named `site.db` and is located within the `App` directory. No separate database server process needs to be installed or configured, which makes getting a development environment running a matter of a single `pip install` command followed by a database initialization step. SQLite fully supports all the SQL features used by SQLAlchemy in this application including foreign keys, joins, and transactions.

A note about SQLite and foreign key enforcement: by default, SQLite does not enforce foreign key constraints unless the `PRAGMA foreign_keys = ON` statement is executed for each connection. SQLAlchemy does not do this automatically for SQLite. In this application, this is not a significant concern because the application logic enforces

referential integrity through the ORM relationships before any database operation is attempted.

5.3.2 MySQL

MySQL is the intended production database. The commented-out MySQL connection string in `__init__.py` shows the format: `mysql://root:root@localhost/sms`. For a production deployment, these credentials would be stored as environment variables rather than hardcoded. MySQL provides better concurrent write handling than SQLite, more robust referential integrity enforcement, better performance under production traffic, and more sophisticated backup and replication options.

The switch from SQLite to MySQL in the application requires changing only the `SQLALCHEMY_DATABASE_URI` configuration value. SQLAlchemy handles all SQL generation, so no application code needs to change. This database-agnostic design is one of the key benefits of using an ORM.

5.3.3 SQL Alchemy ORM

SQLAlchemy is the bridge between the Python application and the database. It maps Python classes to database tables through its declarative base system. Each model class inherits from `db.Model` and defines columns using `db.Column` with type specifications. Relationships between tables are defined using `db.relationship`, which allows navigation from one model to related objects without writing JOIN queries manually.

For example, a `User` object can access all of its requests through `user.requests`, and a `Request` object can access the associated user through `request.user` and the associated stock item through `request.stock`. These relationship traversals are handled by SQLAlchemy lazily by default, meaning the related objects are fetched from the database only when first accessed.

5.3.4 Development and Deployment Tools

TABLE VI
DEVELOPMENT AND DEPLOYMENT TOOLS

Tool	Purpose	Notes
Git	Version control	Source code tracking and collaboration
GitHub	Remote repository hosting	Code review and collaboration platform
Heroku	Cloud deployment platform	Procfile specifies gunicorn web server
pip	Python package manager	requirements.txt lists all dependencies
Virtual Environment	Dependency isolation	conda or python-virtualenv as per README
VS Code	IDE / Code editor	.vscode/settings.json present in repository

CHAPTER 6

SYSTEM DESIGN

6.1 USE CASE DIAGRAM

6.1.1 Actors

The system has three primary actors, each representing a distinct category of user with different responsibilities and permissions within the institution.

- **Regular User:** This actor represents lab assistants, teaching staff, and other college employees who need to request supplies. They interact with the system to browse available items, submit requests, track their request status, and confirm receipt of delivered items.
- **Administrator:** This actor represents department heads or administrative staff responsible for managing the supply approval process. They process incoming requests, manage the inventory catalog, and oversee user accounts.
- **Super User:** This actor represents the highest level of system authority, typically a senior administrative officer. They handle exceptional cases through the special request system and manage role assignments for other users.
- **Email System:** This is a secondary actor representing the external Gmail SMTP service that delivers automated email notifications to users at key events.

6.1.2 Regular User Use Cases

- UC01: Login — User provides email and password;
- UC02: Logout — User ends their session and is redirected to login page
- UC03: Reset Password — User requests reset and sets new password using token
- UC04: Browse Categories — User views all item categories with representative images
- UC05: Browse All Items — User views complete inventory list with remaining quotas
- UC06: Submit Standard Request — User selects item, enters quantity within quota, and submits with message
- UC07: View Request Log — User views all submitted requests with current status and admin comments
- UC08: Confirm Item Receipt — User marks an approved request as received and adds delivery note
- UC09: Submit Special Request — User submits exception request when quota is exhausted
- UC10: View Special Request Log — User views history and status of all special requests
- UC11: Update Profile — User changes name, email, and profile photo after password verification
- UC12: Change Password — User updates password after verifying current password

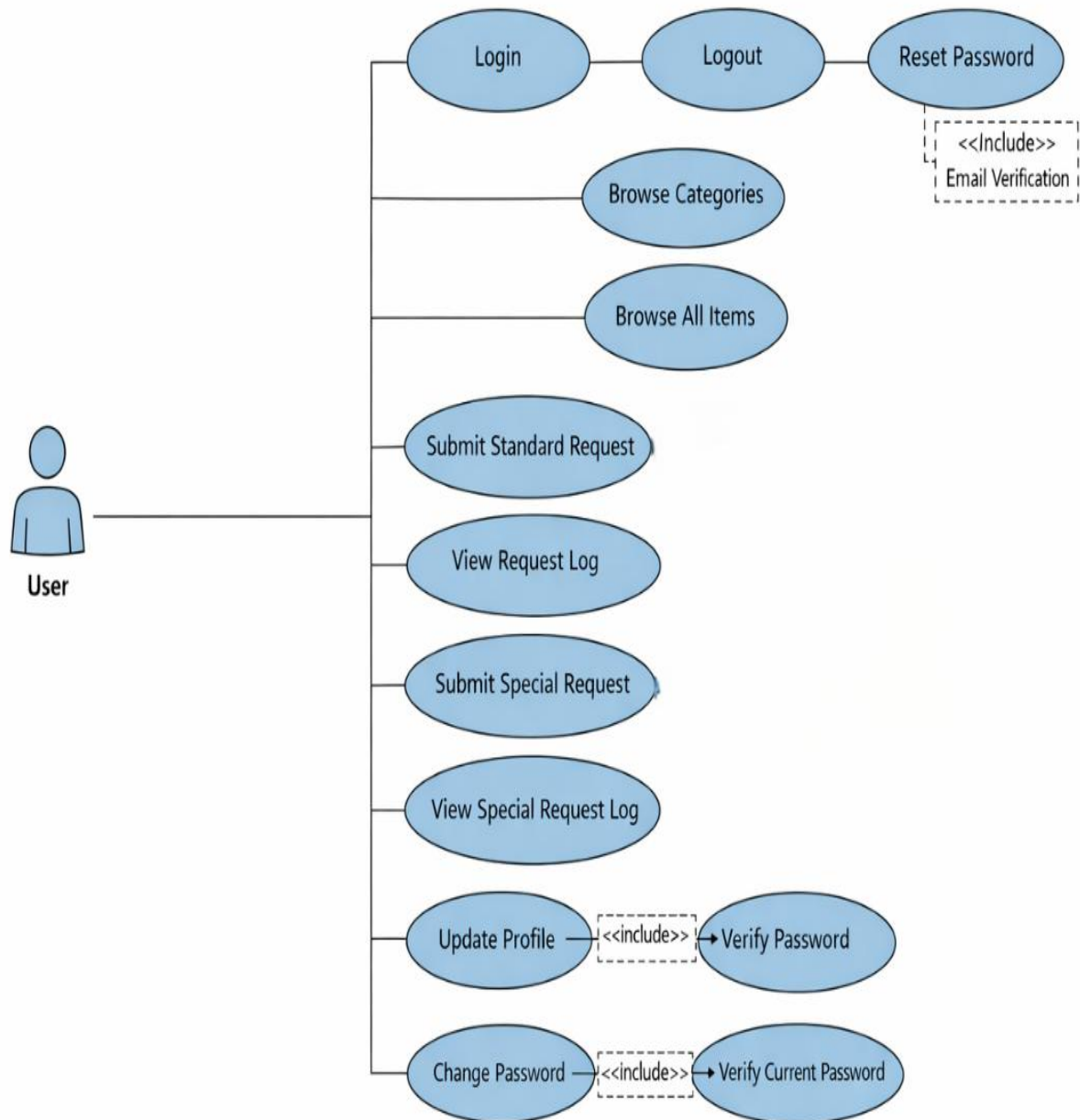


Fig. 6.1: Regular User Use Case

6.1.3 Administrator Use Cases (extends Regular User)

- UC13: View All Pending Requests — Admin sees consolidated queue of all user requests
- UC14: Approve Request — Admin sets approved quantity and comment, updates status to approved
- UC15: Reject Request — Admin records rejection reason, updates status to rejected
- UC16: View Admin Summary — Admin reviews full history of all processed requests
- UC17: View All Stock Items — Admin sees complete inventory with all quantity fields
- UC18: Add Stock Item — Admin creates new inventory item with all required parameters
- UC19: Edit Stock Item — Admin modifies any parameter of an existing stock item
- UC20: View All Categories — Admin sees all item categories with edit options
- UC21: Add Category — Admin creates new category with name and photo
- UC22: Edit Category — Admin modifies category name or representative photo
- UC23: Download Stock CSV — Admin exports complete stock data as CSV file
- UC24: Reset Semester Stock — Admin archives current data and resets present semester counters
- UC25: View All Users — Admin sees list of all registered users with role information
- UC26: Add New User — Admin creates account with auto-generated credentials sent via email

- UC27: View User Details — Admin sees full profile and request history for specific user

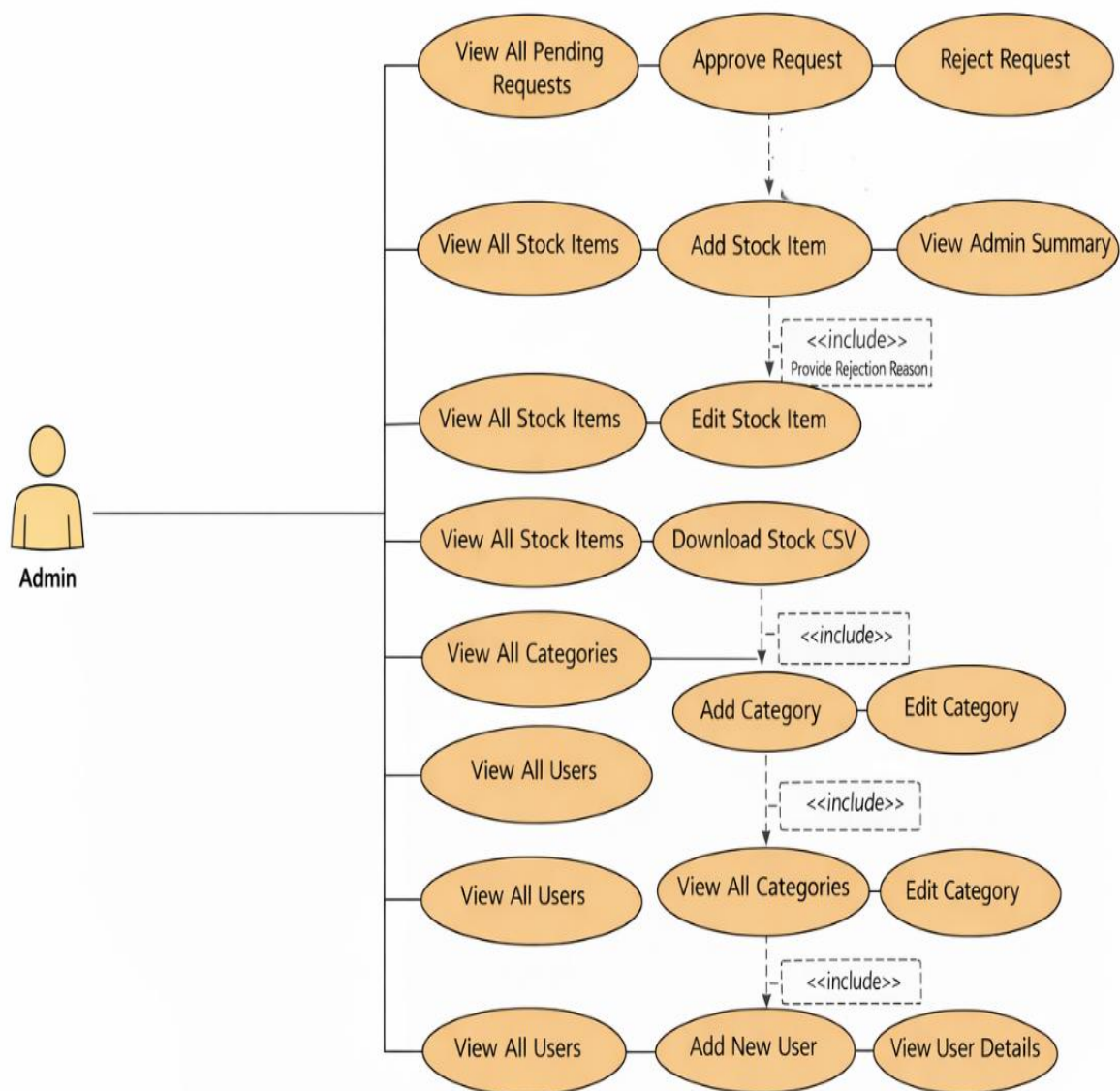


Fig. 6.2: Administrator Use Case

6.1.4 Super User Use Cases (extends Administrator)

- UC28: View Special Request Queue — Super user sees all pending special requests
- UC29: Approve Special Request — Super user approves exception request, creates linked standard request
- UC30: Reject Special Request — Super user rejects exception request with reason
- UC31: View Special Request Summary — Super user sees full history of all special requests
- UC32: Toggle Admin Privileges — Super user grants or revokes admin role for any user
- UC33: Toggle Super User Privileges — Super user grants or revokes super user role for any user
- UC34: Delete User Account — Super user permanently removes a user account with email notification

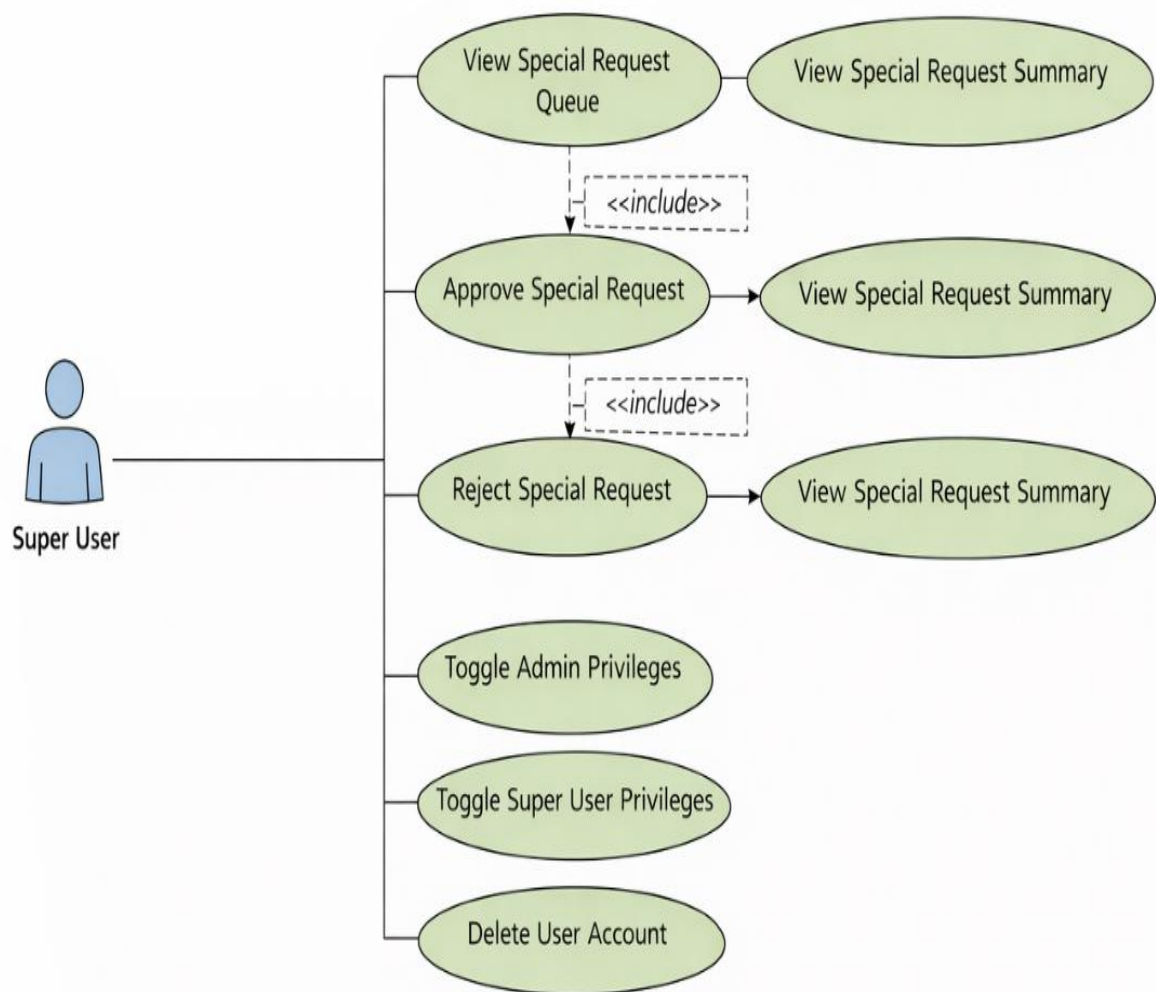


Fig. 6.3: Super User Use Case

6.2 DATA FLOW DIAGRAM

6.2.1 Level 0 DFD — Context Diagram

The context diagram shows the system as a single black box with the external entities that interact with it. Three external entities interact with the system: Users (all roles combined at this level) who send authentication data, request submissions, approval actions, stock management actions, and profile updates into the system; the Email Server (Gmail SMTP) which receives email sending instructions from the system and delivers messages to recipients; and Administrators/Super Users who are a subset of Users but are called out separately because of the outbound reporting data they receive in the form of CSV files and summary views.

6.2.2 Level 1 DFD — Process Decomposition

At the first level of decomposition, the system separates into six major processes, each with their own data flows.

10. Process 1 — Authentication Management: Receives username and password from User. Reads User records from the User data store.
11. Process 2 — Request Lifecycle Management: Receives request submissions from Users. Writes new Request records to the Request data store. Returns confirmation or error messages to Users.
12. Process 3 — Approval Workflow Management: Receives approval and rejection actions from Administrators. Reads Request records to validate the action. Updates Request records with new status, quantity, comment, and processed-by fields. Returns updated status views to both Administrators and requesting Users.
13. Process 4 — Inventory Management: Receives stock addition, edit, and reset actions from Administrators. Reads and writes Stock and Category data stores. Generates CSV output to Administrators. Resets semester counters.
14. Process 5 — User Account Management: Receives user creation and deletion actions from Administrators and Super Users. Reads and writes User data store.

15. Process 6 — Special Request Management: Receives special request submissions from Users. Validates against Stock and Request data stores. Writes SpecialRequest records. Receives approval actions from Super Users. Creates linked Request records on approval.

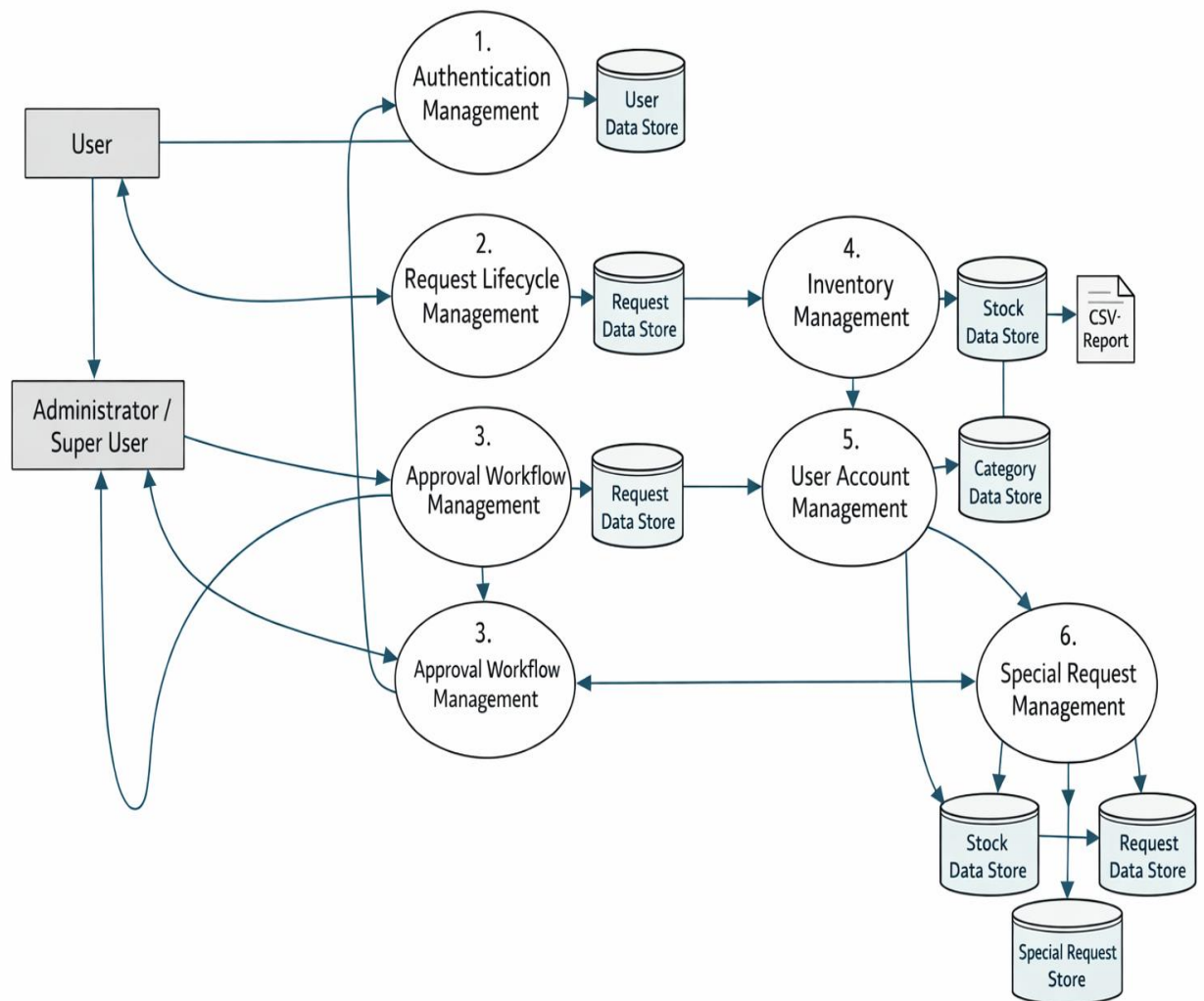


Fig. 6.4: Data flow Diagram

6.3 ER DIAGRAM

Entity Descriptions and Attributes

TABLE VII
ENTITY DESCRIPTION AND ATTRIBUTES

Entity	Attribute	Data Type	Constraint	Description
User	id	Integer	PK, Auto-increment	Unique user identifier
User	email	String(250)	Unique, Not Null	Login credential and contact address
User	first_name	String(255)	Not Null	User's first name for display
User	last_name	String(255)	Not Null	User's last name for display
User	password	String(60)	Not Null	Bcrypt hash of user's password
User	isAdmin	Boolean	Default: False	Administrator role flag
User	isSuperUser	Boolean	Default: False	Super user role flag
User	picture	String(60)	Default: 'default.jpg'	Profile photo filename
Category	id	Integer	PK, Auto-increment	Unique category identifier

Entity	Attribute	Data Type	Constraint	Description
Category	name	String(255)	Not Null	Category display name
Category	picture	String(60)	Default: 'default.jpg'	Category photo filename
Stock	id	Integer	PK, Auto-increment	Unique stock item identifier
Stock	category_id	Integer	FK: Category.id	Parent category reference
Stock	item	String(255)	Not Null	Item display name
Stock	qty_prev	Integer	Not Null	Consumption in previous semester
Stock	avail	Integer	Not Null	Current available quantity in store
Stock	qty_req	Integer	Not Null	Quantity required for procurement
Stock	qty_pres	Integer	Default: 0	Consumption in current semester
Stock	minimum_limit	Integer	Not Null	Minimum stock threshold
Stock	maximum_limit	Integer	Not Null	Maximum stock threshold
Stock	quota	Integer	Not Null	Per-user maximum request per semester

Entity	Attribute	Data Type	Constraint	Description
Request	id	Integer	PK, Auto-increment	Unique request identifier
Request	user_id	Integer	FK: User.id, Not Null	Requesting user reference
Request	stock_id	Integer	FK: Stock.id, Not Null	Requested item reference
Request	original_quantity	Integer	Not Null	Quantity originally requested
Request	qty	Integer	Not Null	Quantity approved by administrator
Request	date_applied	DateTime	Default: now()	Submission timestamp
Request	status	Integer	Default: 0	0=pending, 1=approved, -1=rejected
Request	accepted	Boolean	Default: False	Delivery confirmation flag
Request	admins_comment	Text	Default: 'No Comments'	Administrator's response note
Request	users_comment	Text	Not Null	User's request justification
Request	received_comment	Text	Default: 'No Comments'	Delivery acknowledgment note

Entity	Attribute	Data Type	Constraint	Description
Request	processed_by	String(255)	Default: 'Not yet Processed'	Administrator's display name
SpecialRequest	id	Integer	PK, Auto-increment	Unique special request identifier
SpecialRequest	user_id	Integer	FK: User.id, Not Null	Requesting user reference
SpecialRequest	stock_id	Integer	FK: Stock.id, Not Null	Requested item reference
SpecialRequest	original_quantity	Integer	Not Null	Quantity originally requested
SpecialRequest	qty	Integer	Not Null	Quantity approved by super user
SpecialRequest	date_applied	DateTime	Default: now()	Submission timestamp
SpecialRequest	status	Integer	Default: 0	0=pending, 1=approved, -1=rejected
SpecialRequest	admins_comment	Text	Default: 'No Comments'	Super user's response note
SpecialRequest	users_comment	Text	Not Null	User's justification for exception
SpecialRequest	processed_by	String(255)	Default: 'Not yet Processed'	Super user's display name

Relationships

- User to Request: One-to-Many. One user can have many requests. Each request belongs to exactly one user. Implemented as a foreign key `user_id` on the Request table and a backref 'user' on the relationship.
- User to SpecialRequest: One-to-Many. One user can have many special requests. Each special request belongs to exactly one user.
- Category to Stock: One-to-Many. One category can contain many stock items. Each stock item belongs to exactly one category. Implemented as a foreign key `category_id` on the Stock table.
- Stock to Request: One-to-Many. One stock item can appear in many requests from different users or at different times. Each request is for exactly one stock item.
- Stock to SpecialRequest: One-to-Many. Same relationship pattern as Stock to Request but for the special request table.

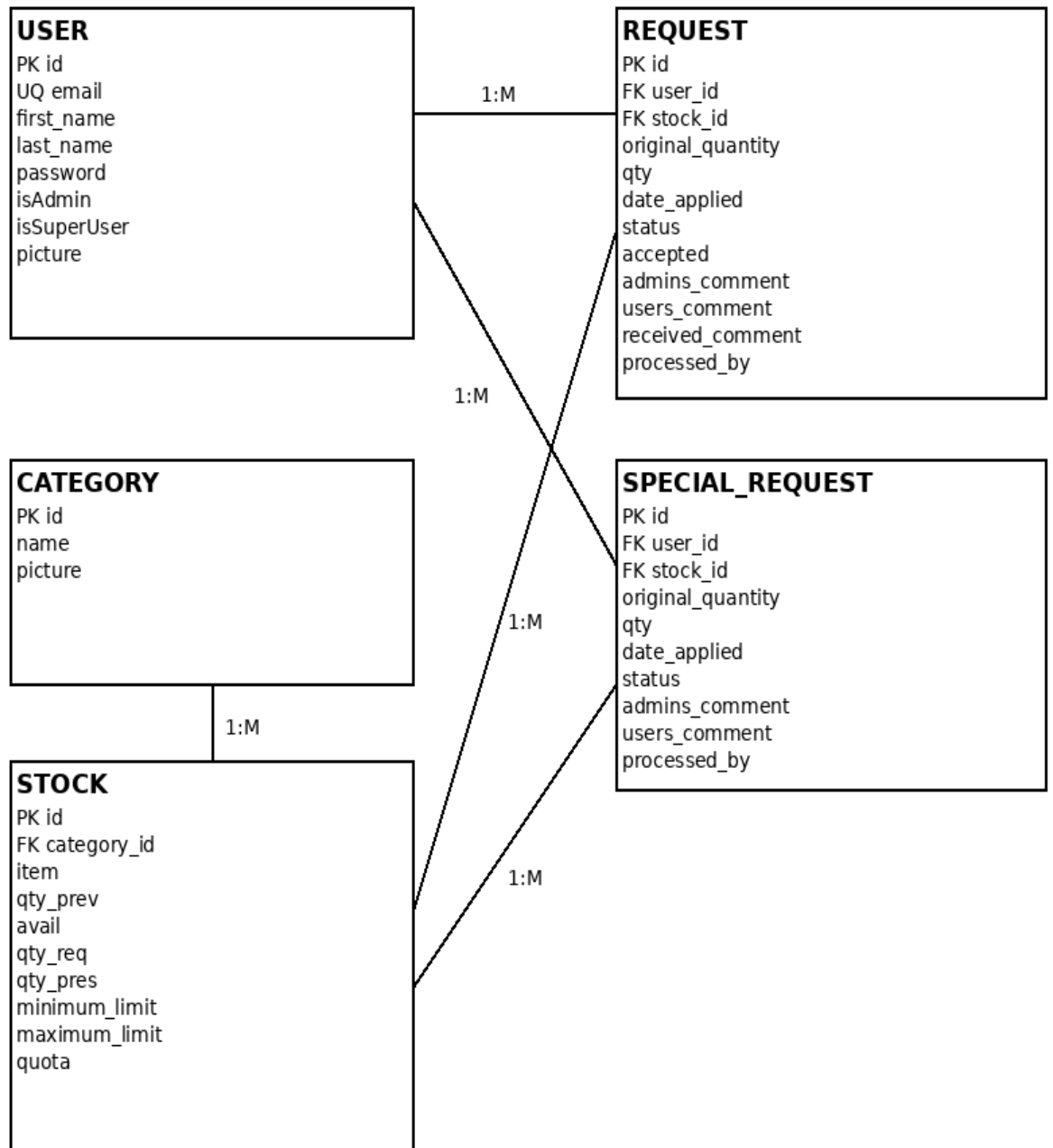


Fig. 6.5: E-R Diagram

6.4 UML DIAGRAMS

6.4.1 Class Diagram Description

The class diagram for this application maps directly to the SQLAlchemy models in `models.py`. Five classes are defined, each corresponding to a database table.

The User class has attributes `id`, `email`, `first_name`, `last_name`, `password`, `isAdmin`, `isSuperUser`, and `picture`. It has two methods: `get_reset_token()` which generates a signed token for password reset using the application secret key, and `verify_reset_token(token)` which is a static method that decodes a token and returns the corresponding User or None. User has associations to Request (one-to-many) and SpecialRequest (one-to-many).

The Category class has attributes `id`, `name`, and `picture`. It has an association to Stock (one-to-many). The Category class has no methods of its own beyond the `__repr__` for debugging.

The Stock class has attributes `id`, `category_id`, `item`, `qty_prev`, `avail`, `qty_req`, `qty_pres`, `minimum_limit`, `maximum_limit`, and `quota`. It has associations to both Request and SpecialRequest (one-to-many each), and to Category (many-to-one through `category_id`).

The Request class has attributes `id`, `user_id`, `stock_id`, `original_quantity`, `qty`, `date_applied`, `status`, `accepted`, `admins_comment`, `users_comment`, `received_comment`, and `processed_by`. It has many-to-one associations to User and Stock.

The SpecialRequest class mirrors Request in structure with slightly fewer fields, omitting `accepted` and `received_comment` since the special request pathway does not include a delivery confirmation step directly.

6.4.2 Sequence Diagram: User Login Flow

- User opens browser and navigates to /login URL
- Browser sends HTTP GET /login to Flask server
- Flask routes to login() handler, checks current_user.is_authenticated
- User is not authenticated, so login template is rendered and returned
- Browser renders login form with email and password fields
- User enters credentials and submits form
- Browser sends HTTP POST /login with form data
- Flask validates CSRF token via Flask-WTF
- LoginForm validates field requirements (DataRequired, Email)
- Handler queries User.query.filter_by(email=form.email.data).first()
- If user found, bcrypt.check_password_hash(user.password, form.password.data)
- If password matches, login_user(user) creates session
- Flash welcome message and redirect to home or next_page
- Browser follows redirect to /
- home() handler renders home.html with current_user context

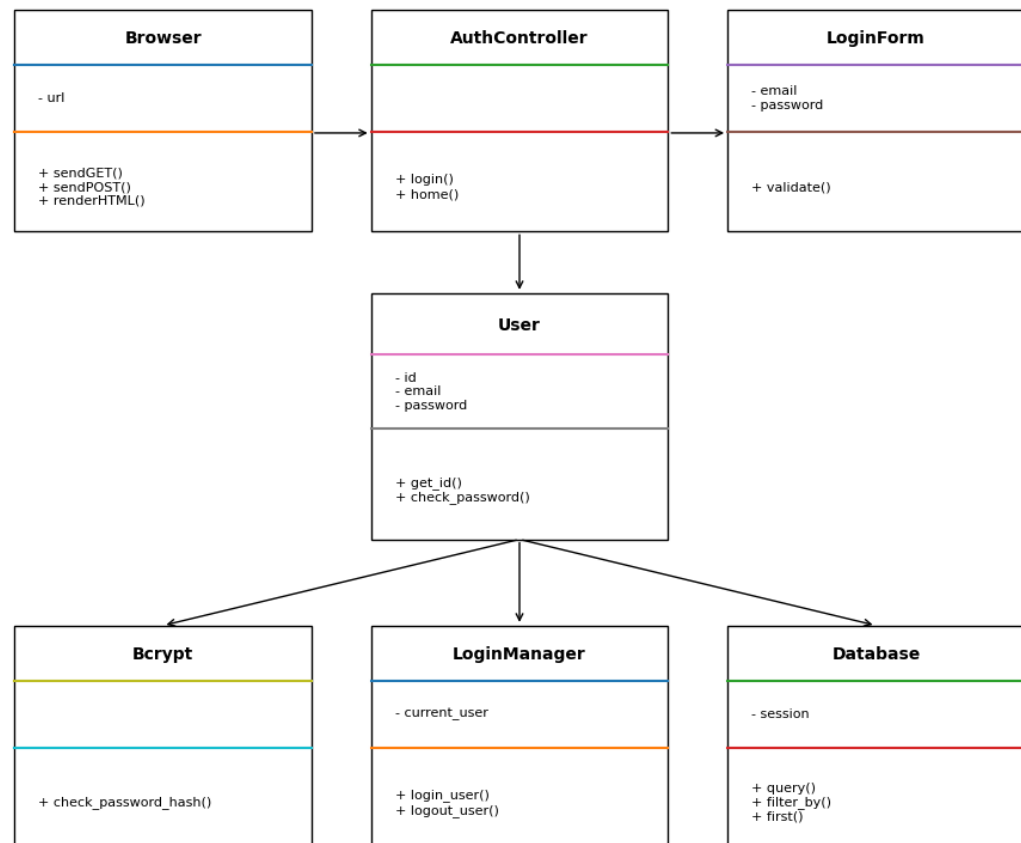


Fig. 6.6: User Login

6.4.3 Sequence Diagram: Standard Request Submission

- User navigates to `/user/home` or `/categories/category_id`
- Server queries all Stock items, calculates `quota_left` for each for current user
- Template renders stock table with `quota_left` displayed per item
- User clicks Request button for a specific item
- Browser navigates to `/make/request/stock_id` (GET)
- Server fetches Stock by `stock_id`, renders `request_stock.html` with RequestForm
- User fills in quantity and message, clicks Confirm Request
- Browser sends POST `/make/request/stock_id` with form data

- Server validates form, queries existing requests for user and stock
- Server calculates total used = sum of qty where status in (0, 1)
- Server calculates quota_left = max(0, stock.quota - total_used)
- If form.quantity_req.data > quota_left: flash error, re-render form
- Else: create Request(user_id, stock_id, qty, users_comment, original_quantity)
- db.session.add(request), db.session.commit()
- Flash success message, re-render form

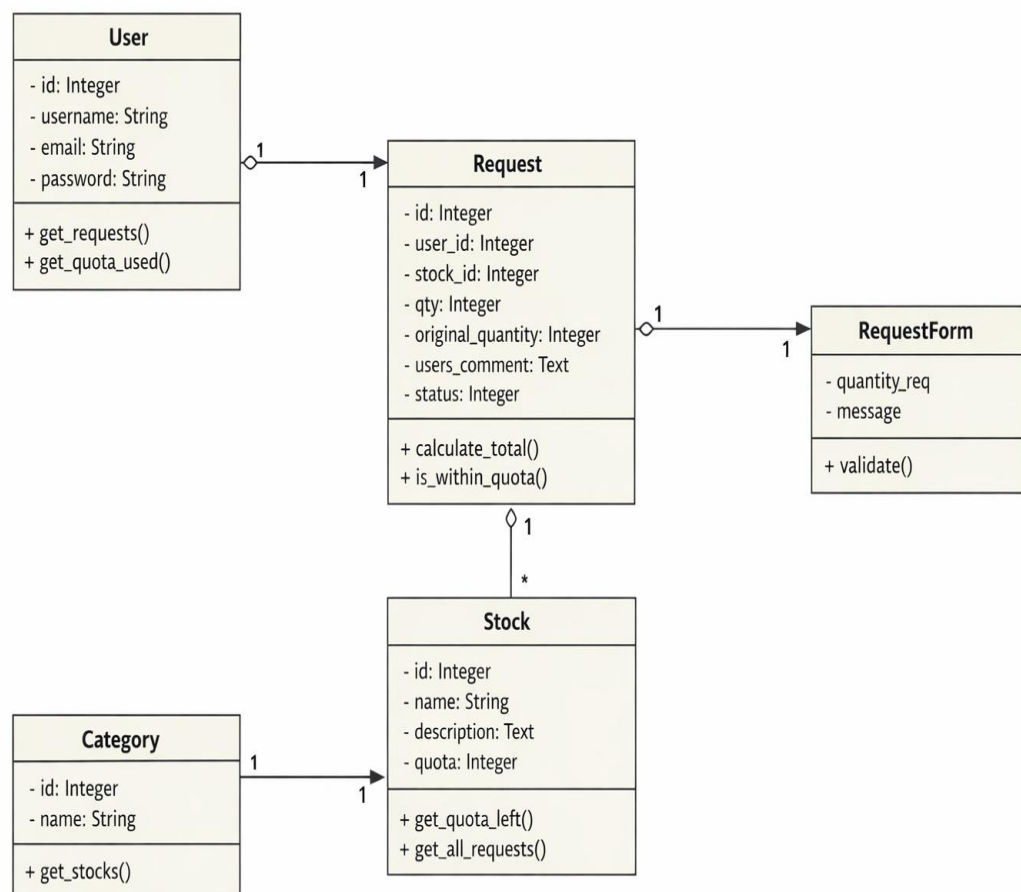


Fig. 6.7: Request submission

CHAPTER 7

IMPLEMENTATION

7.1 FRONTEND IMPLEMENTATION

7.1.1 Template Architecture

The entire frontend is built using Jinja2's template inheritance system. A single base file, `layout.html`, defines the complete page structure that all other templates share. The HTML head section in `layout.html` imports Bootstrap 4 CSS from MaxCDN, Google Fonts (Ubuntu family), Font Awesome icons, jQuery, Popper.js, Bootstrap JavaScript, and the application's own CSS and JavaScript files. This means updates to any globally used resource only need to be made in `layout.html` rather than in every individual page template.

The `layout.html` body contains three major sections. The header section contains the navigation bar and the flash message display area. The main content block uses Jinja2's block syntax: `{% block content %}` `{% endblock %}`, which child templates override to inject their specific page content. The footer section contains the institution logo, quick navigation links, team member names with LinkedIn profile links, and legal boilerplate links.

7.1.2 Navigation Bar Implementation

The navigation bar uses Bootstrap's navbar component with the `navbar-expand-lg` class, making it collapsible on screens smaller than the large breakpoint (992px). On mobile devices, the bar collapses to show only the brand logo and a hamburger toggle button. Clicking the toggle reveals the full navigation menu.

The navigation items are conditionally rendered using Jinja2 if statements. The Make Request dropdown and Request Log dropdown are shown to all authenticated users. The Requests link, Stocks dropdown, and Admin Summary link are conditionally rendered inside `{% if current_user.isAdmin %}` blocks. The Special Requests dropdown is rendered inside `{% if current_user.isSuperUser %}` blocks. The Profile dropdown with avatar image and Signout link is always shown to authenticated users.

The signout functionality uses a hidden HTML form with a POST method rather than a GET link. This is intentional: logout should be a POST action to prevent cross-site request forgery attacks where a malicious page could embed an image whose URL triggers a GET logout, inadvertently logging out the user. The Signout link calls a JavaScript function that programmatically submits this hidden form.

7.1.3 Stock Management Interface

The stock management page (stocks.html) presents all inventory items in a striped, bordered table with a real-time search field above it. The table has eleven columns: serial number, item name, category, previous semester quantity, stock in hand, quantity required, present semester quantity, maximum limit, minimum limit, quota, and an action column with an Edit button.

The Add New Item button triggers a Bootstrap modal dialog rather than navigating to a separate page. The modal contains a form with fields for item name, category dropdown (populated server-side from all Category records), available quantity, required quantity, quota, minimum quantity, and maximum quantity. Submitting the modal form posts to `/admin/stocks/add`. Using a modal for this keeps the user on the stock management page and provides a better user experience than a page navigation.

The Download and Reset buttons in the table footer submit POST forms to their respective routes. The Download button submits to `/admin/stock/download` and triggers a CSV file download. The Reset button submits to `/admin/stock/reset` which performs the

semester reset and also triggers the CSV download simultaneously, ensuring the data is backed up before the reset occurs.

7.1.4 Request Summary Interface

The user's request summary page presents their request history in a horizontally scrollable table to accommodate the large number of columns without breaking the layout on smaller screens. The table columns are: request ID, item name, requested quantity, accepted quantity, submission date (formatted as YYYY-MM-DD using Jinja2's `strftime` filter), status icon, received button, comment textarea, processed-by name, and admin's comment textarea.

The status is displayed as a Font Awesome icon rather than text for visual clarity. A green check circle icon (`fa-check-circle`) indicates approved status. A red times icon (`fa-times`) indicates rejected status. A black clock icon (`fa-clock`) indicates pending status. These icons communicate status at a glance without requiring the user to read text.

The Received button is rendered as an active clickable button only when the request has status 1 (approved) and accepted is False. In all other states including pending, rejected, and already-confirmed requests, the button is rendered as a disabled secondary button. The confirmation textarea is rendered as an active input only for the same condition; otherwise it is a disabled textarea showing the stored `received_comment` value.

7.1.5 CSS Custom Styling

Beyond Bootstrap's defaults, the application defines custom styles in several CSS files. The login page uses a full-viewport-height flexbox layout with the login form centered both vertically and horizontally. The background image is applied to the body element using CSS background shorthand with center center fixed positioning to prevent it from scrolling with the page content.

The `.bgcolor` and `.colors` classes use the dark red (`#9d0b0b`) brand color and are applied to buttons, card backgrounds, and text accents throughout the interface. The navbar CSS applies the light blue-grey (`#e8f1f5`) background to the navigation bar. The

font family Ubuntu (imported from Google Fonts) is applied to h4 elements for visual variety, while the rest of the interface uses the browser's default sans-serif as specified by Bootstrap.

Responsive breakpoints in the login CSS hide certain elements on mobile screens using `@media (max-width: 600px)` queries that set `display: none` on department name and college logo elements, replacing them with smaller text using the `.responsive` and `.clgname` classes.

7.2 BACKEND IMPLEMENTATION

7.2.1 Application Initialization

The Flask application is initialized in `App/__init__.py`. This file creates the Flask app instance with `Flask(__name__)`, sets the `SECRET_KEY` configuration (a hardcoded hex string that should be moved to an environment variable in production), and sets the `SQLALCHEMY_DATABASE_URI` to point to the SQLite database file. It then initializes all extensions by passing the app instance: `SQLAlchemy(app)` creates the db object, `Bcrypt(app)` creates the `bcrypt` object, `LoginManager(app)` creates the `login_manager`, and `Mail(app)` creates the mail object.

The `login_manager` configuration sets `login_view` to 'login', which is the name of the route function that handles the login page. This tells Flask-Login where to redirect unauthenticated users who try to access protected routes. The `login_message_category` is set to 'info' so that the 'please log in' message is styled as an info alert rather than a default alert.

The mail configuration sets `MAIL_SERVER` to `smtp.googlemail.com`, `MAIL_PORT` to `587`, `MAIL_USE_TLS` to `True`, and leaves `MAIL_USERNAME` and `MAIL_PASSWORD` as empty strings that should be filled with valid Gmail credentials before deployment. The final import line imports routes from the `App` package, which registers all route handlers with the app.

7.2.2 Route Handler Organization

The routes.py file contains all URL route handlers organized into four sections. The Admin routes section handles all URLs prefixed with /admin/ and contains handlers for stock management, category management, request processing, and user management. Each handler in this section begins with the pattern: if not current_user.isAdmin: abort(403). The User routes section handles request submission, special request submission, and user-facing views. The General routes section handles the home page, profile management, authentication, and help pages. A utilities section at the bottom defines helper functions that are called by the route handlers.

TABLE VIII
ROUTE HANDLER

Route	Method	Handler Function	Access Level
/admin/stocks	GET, POST	stocks()	Admin
/admin/stocks/edit/<id>	GET, POST	edit_stock(stock_id)	Admin
/admin/stocks/add	POST	add_stocks()	Admin
/admin/stock/download	POST	download()	Admin
/admin/stock/reset	GET	reset()	Admin
/admin/categories	GET	admin_categories()	Admin
/admin/categories/<id>	GET	admin_category(category_id)	Admin
/admin/category/add	POST	add_category()	Admin
/admin/category/edit/<id>	GET, POST	edit_category(category_id)	Admin

Route	Method	Handler Function	Access Level
/admin/requests	GET	admin_request()	Admin
/admin/request/accept/<id>	POST	accept_request(req_id)	Admin
/admin/request/delete/<id>	POST	reject_request(req_id)	Admin
/admin/requests/summary	GET	admin_summary()	Admin
/admin/specialrequests	GET	admin_special_request()	Super User
/admin/specialrequest/accept/<id>	POST	accept_special_request(req_id)	Super User
/admin/specialrequest/delete/<id>	POST	reject_special_request(req_id)	Super User
/admin/specialrequests/summary	GET	admin_special_summary()	Super User
/admin/users	GET	display_users()	Admin
/admin/users/add	GET, POST	add_users()	Admin
/user/home	GET, POST	user_home()	All users
/make/request/<id>	GET, POST	make_request(stock_id)	All users
/user/requests/received/<id>	POST	request_received(request_id)	All users

Route	Method	Handler Function	Access Level
/categories	GET	categories()	All users
/categories/<id>	GET	category(category_id)	All users
/user/request/summary	GET	user_summary()	All users
/user/specialrequest/<id>	GET, POST	make_special_request(stock_id)	All users
/user/specialrequests/summary	GET	user_special_summary()	All users
/	GET	home()	All users
/login	GET, POST	login()	Public
/logout	POST	logout()	All users
/reset_password	GET, POST	reset_request()	Public
/reset_password/<token>	GET, POST	reset_token(token)	Public
/profile	GET	account()	All users
/profile/update	GET, POST	profile()	All users

Route	Method	Handler Function	Access Level
/profile/update/password	GET, POST	update_password()	All users
/help	GET	help()	All users

7.2.3 Form Definitions and Validation

TABLE IX
FORM DEFINITION AND VALIDATION

Form Class	Fields	Key Validators
LoginForm	email, password	Email(), DataRequired(), Length(min=4, max=20)
RegistrationForm	email, first_name, last_name	Email(), DataRequired(), Length constraints
UpdatePassword	prev_password, password, confirm_password	EqualTo('password'), Length(min=4, max=20)
RequestForm	quantity_req, message	DataRequired(), Length(max=100) on message
EditStocks	stock_name, avail, qty_req, category, maximum_limit, minimum_limit, quota	DataRequired() on all fields, SelectField for category
ProfileForm	first_name, last_name, email, password, picture	Email(), FileAllowed(['jpg', 'png']) on picture
EditCategoryForm	name, picture	DataRequired(), FileAllowed(['jpg', 'png'])

Form Class	Fields	Key Validators
RequestResetForm	email	Email(), custom validate_email() checks user exists
ResetPasswordForm	password, confirm_password	EqualTo('password'), DataRequired()

7.2.4 Image Processing Pipeline

Profile photos and category images go through a standardized processing pipeline when uploaded. The `save_picture()` utility function accepts the uploaded file object and a folder name (either 'profile' or 'category') as parameters. It first generates a cryptographically random eight-byte hexadecimal string using `secrets.token_hex(8)` which becomes the base of the new filename. It then extracts the file extension from the original filename using `os.path.splitext()` and constructs the new filename by combining the hex string with the original extension.

The full path for saving is constructed using `os.path.join()` with `app.root_path` as the base and the appropriate subdirectory within `static/images/`. The image is then opened using `Image.open()` from Pillow, a thumbnail is created at 250x250 pixels using the `thumbnail()` method which preserves the aspect ratio while fitting within the dimensions, and the result is saved to the full path. The function returns just the filename (not the full path) which is stored in the database record.

This approach has several benefits. Renaming files to random hexadecimal strings prevents filename collisions between different users uploading files with the same name. Resizing all images to a consistent maximum dimension prevents large uploads from consuming excessive storage. Storing only the filename rather than the full path in the database makes the application portable across different deployment environments.

7.3 DATABASE DESIGN

7.3.1 Schema Implementation

The database schema is implemented through SQLAlchemy's declarative base system. Each model class inherits from `db.Model`. Column definitions use `db.Column()` with the data type as the first argument. Constraints like `nullable=False`, `unique=True`, and default values are specified as keyword arguments. Foreign key references use `db.ForeignKey()` with the `table.column` format. Relationships between tables use `db.relationship()` with a `backref` argument to create a reverse reference on the related model.

The User model also inherits from `UserMixin` provided by Flask-Login. This mixin provides default implementations of the `is_authenticated`, `is_active`, `is_anonymous`, and `get_id` methods that Flask-Login requires on user model objects. Without this mixin, these methods would need to be implemented manually.

7.3.2 Database Initialization

The database tables are created from the model definitions using `db.create_all()` called within an application context. In Flask, database operations must be performed within an application context because they need access to the app configuration. The `db.create_all()` method inspects all registered model classes and creates any tables that do not already exist in the configured database. This means it is safe to call on an existing database: it will create missing tables but will not modify or drop existing ones.

7.3.3 Query Patterns

The application uses SQLAlchemy's query API for all database operations. Several common query patterns appear throughout the codebase. `Model.query.all()` retrieves all records of a type. `Model.query.get(id)` retrieves a single record by primary key. `Model.query.filter_by(column=value).all()` retrieves records matching a single condition. `Model.query.filter_by(column=value).first()` retrieves the first matching record or `None`.

`Model.query.get_or_404(id)` retrieves by primary key and aborts with a 404 response if not found, which is used throughout the admin routes for record lookups.

New records are created by instantiating the model class with keyword arguments for all required fields, calling `db.session.add(instance)`, and then `db.session.commit()`. Updates are made by retrieving the object, modifying its attributes directly, and calling `db.session.commit()`. Deletions use `db.session.delete(instance)` followed by `db.session.commit()`. All three of these operations benefit from SQLAlchemy's transaction handling, which ensures that either all changes in a session are committed or none of them are, preventing partial updates.

CHAPTER 8

APPLICATION DEMONSTRATION & SCREENSHOTS

8.1 USER INTERFACE SCREENS

8.1.1 Login Page

The login page is the application's entry point and serves as the first impression for all users. The page uses a full-viewport background photograph that creates a sense of place appropriate for an educational institution. Over this background, a semi-transparent centered card presents the login form. At the top of the card, the institution logo is displayed prominently to reinforce institutional identity. Below the logo, the system name is shown in a heading.

The form itself is intentionally minimal: an email address field, a password field, and a Login button styled in the application's dark red brand color. Below the button, a small link reads 'Forgot Password?' and leads to the password reset flow. The form fields use HTML5 type attributes which trigger appropriate keyboards on mobile devices and provide basic browser-level validation before the form is submitted to the server.

On successful login, the user is greeted with a flash message 'Welcome [first name]!' and redirected to the home page. On failure, a red alert message explains that the login credentials did not match, without specifying whether the email or password was incorrect (a security best practice that prevents enumeration attacks).

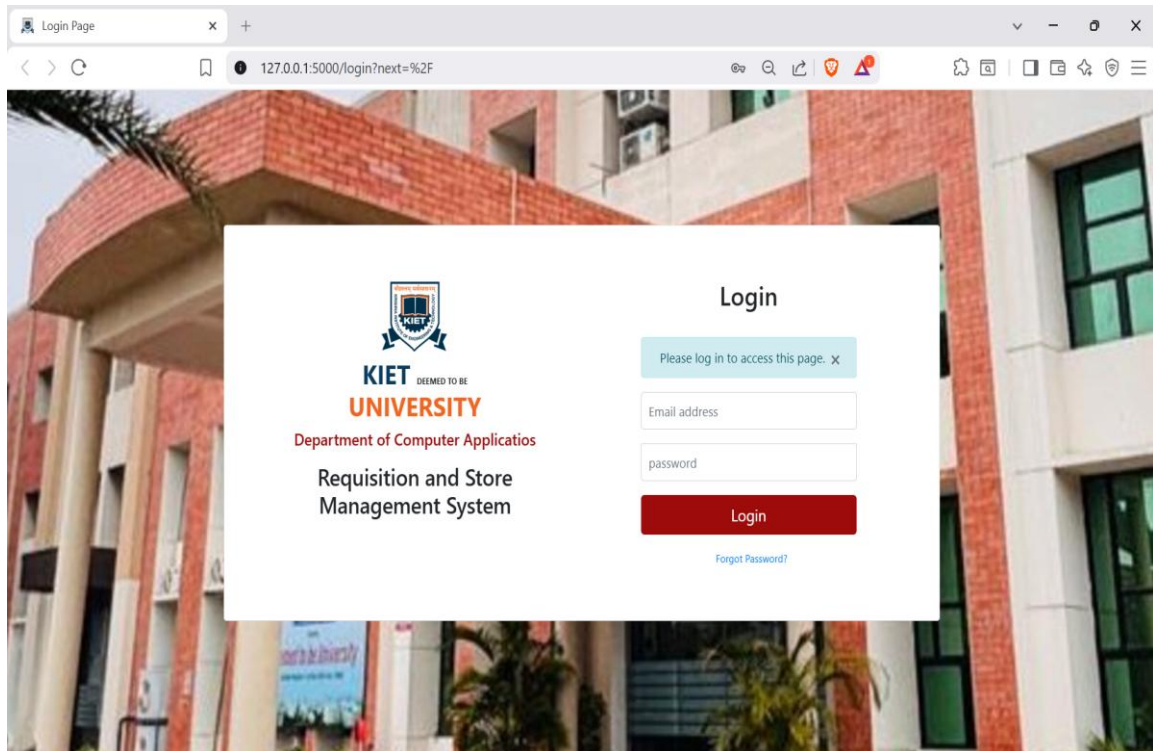


Fig. 8.1: Login Page

8.1.2 Home Page — Regular User View

The home page presents a personalized welcome message showing the user's full name and a grid of quick-access cards. For a regular user, three cards are shown: Make a Request (linking to the categories page), Requests Log (linking to the request summary), and Account (linking to the profile page). A fourth Help card links to the built-in user manual. The cards use the dark red brand color with white text on the card header areas.

This minimal home page is intentional. Regular users have a small number of actions they need to perform, and surfacing all of them immediately without requiring navigation reduces the learning curve significantly. New users can understand the entire user-facing workflow just by reading the home page cards.

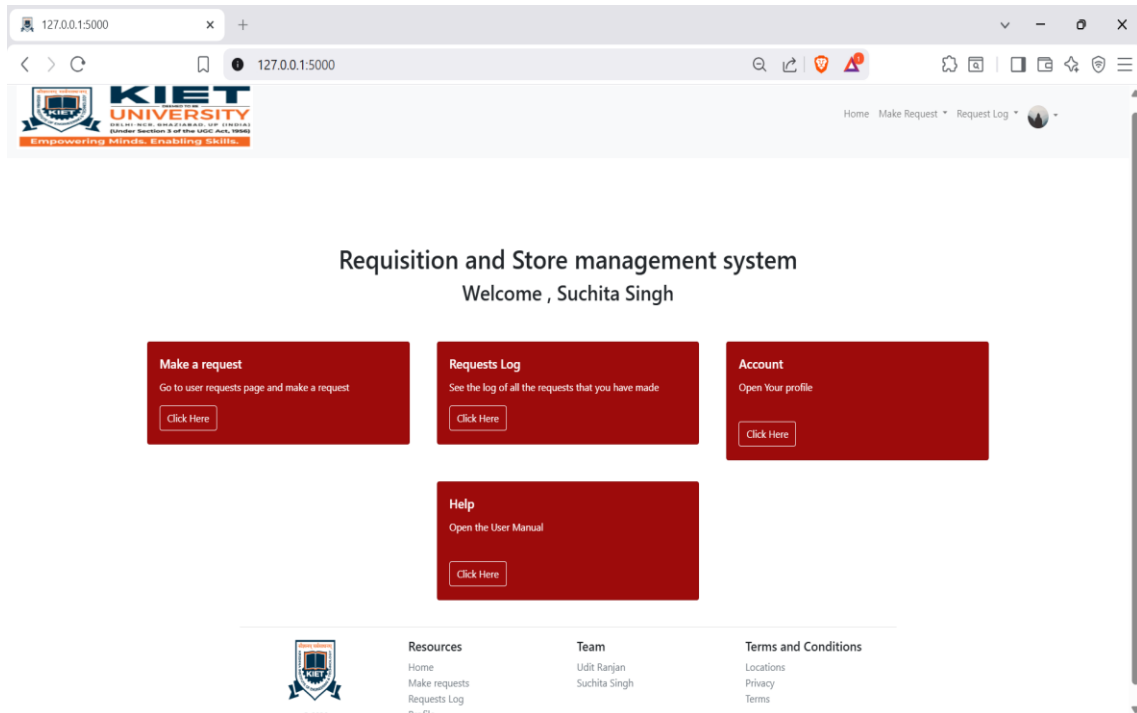


Fig. 8.2: Normal user home Page

8.1.3 Home Page — Administrator View

When an administrator logs in, the home page shows additional cards beyond those visible to regular users: Requests (linking to the pending requests queue), Open Stocks (linking to the full inventory management page), Admin Summary (linking to the processed requests history), and Manage Users (with separate buttons for viewing and adding users). These extra cards appear because of the Jinja2 conditional rendering that checks `current_user.isAdmin`.

The visual treatment of admin cards is identical to regular user cards, reinforcing that administrators can still perform all regular user actions in addition to their management responsibilities. There is no visual hierarchy that makes the admin actions feel more important than the user actions; they all coexist in the same card grid.

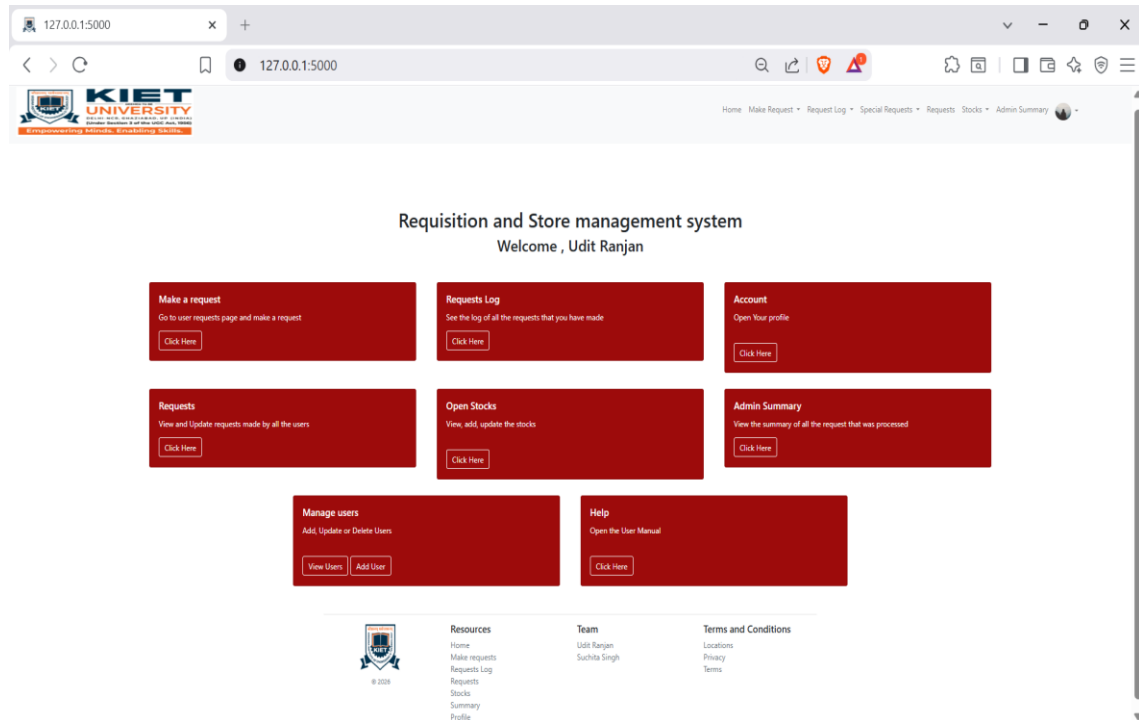


Fig. 8.3: Admin user home Page

8.1.4 Categories Page

The categories page presents each category as a visual card with the category's representative photograph as the card header image and the category name as the card body. Clicking on a category card navigates to the filtered stock view showing only items in that category. This visual approach to category navigation is more intuitive than a plain list and helps users who are not sure of the exact category name for the item they need.

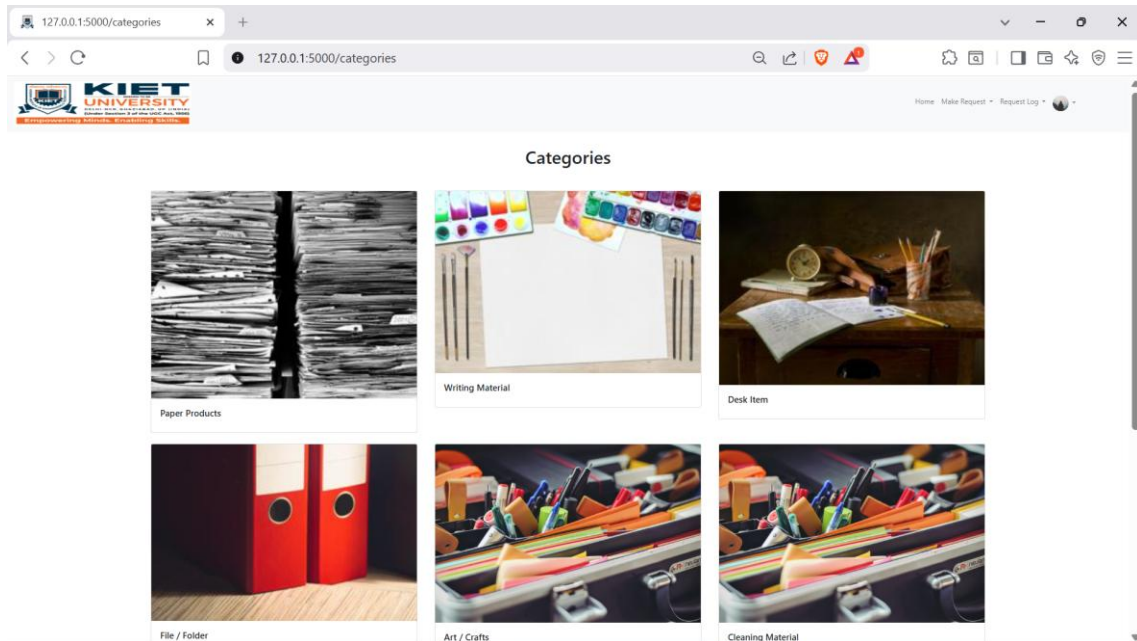


Fig. 8.4: Category Page - I

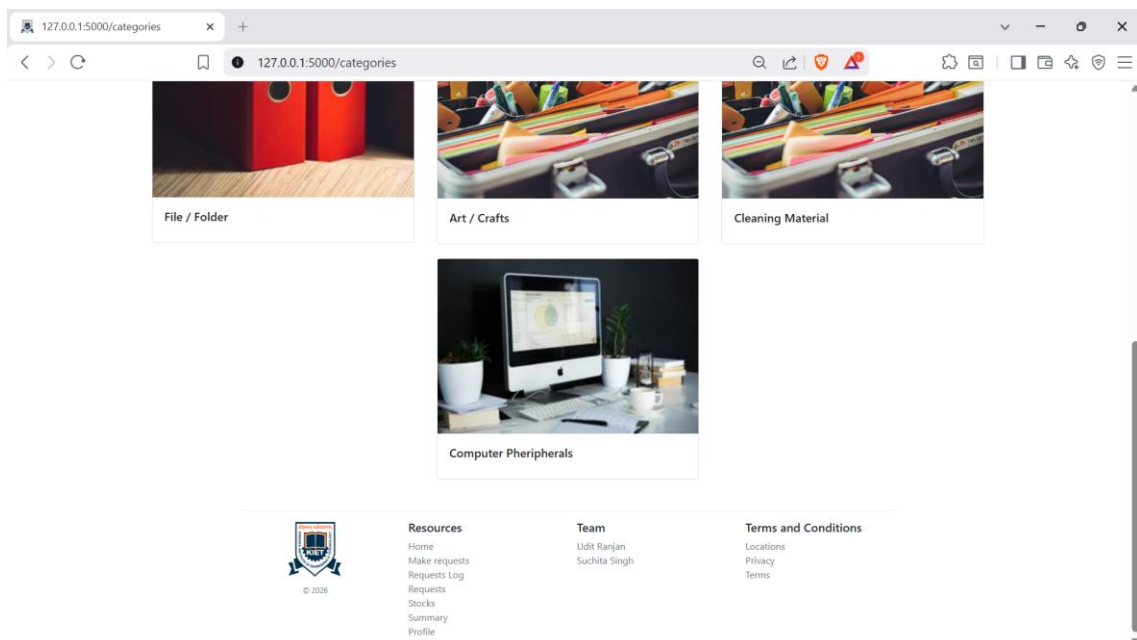
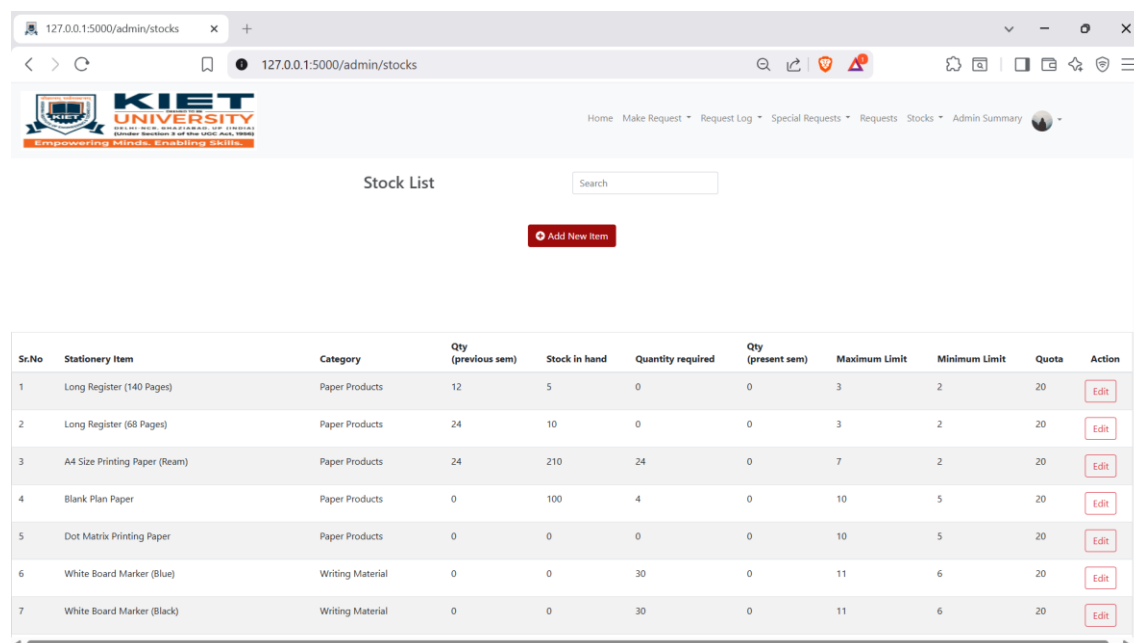


Fig. 8.5: Category Page - II

8.1.5 Stock Listing Page — User View

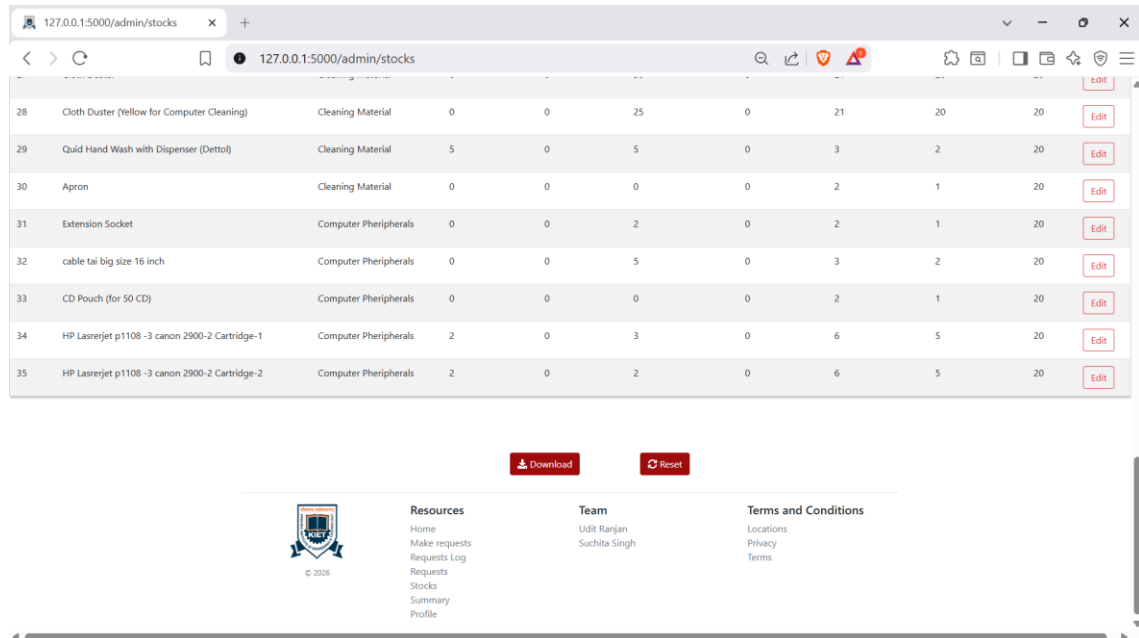
When a user browses the complete item list or a filtered category view, each item is displayed in a table row showing the item name, category, available quantity, quota limit, remaining quota for the current user, and a Request button. If the user's remaining quota for an item is zero, the Request button is replaced by a Special Request button. Items where the user has no remaining quota and has therefore consumed their full allocation are visually distinct.

The real-time search box above the table allows users to type any part of an item name and see only matching rows instantly. This is particularly valuable when the institution has a large number of stocked items across multiple categories and the user knows the name of what they need but not its category.



Sr.No	Stationery Item	Category	Qty (previous sem)	Stock in hand	Quantity required	Qty (present sem)	Maximum Limit	Minimum Limit	Quota	Action
1	Long Register (140 Pages)	Paper Products	12	5	0	0	3	2	20	Edit
2	Long Register (68 Pages)	Paper Products	24	10	0	0	3	2	20	Edit
3	A4 Size Printing Paper (Ream)	Paper Products	24	210	24	0	7	2	20	Edit
4	Blank Plan Paper	Paper Products	0	100	4	0	10	5	20	Edit
5	Dot Matrix Printing Paper	Paper Products	0	0	0	0	10	5	20	Edit
6	White Board Marker (Blue)	Writing Material	0	0	30	0	11	6	20	Edit
7	White Board Marker (Black)	Writing Material	0	0	30	0	11	6	20	Edit

Fig. 8.6: Stock list Page -I



28	Cloth Duster (Yellow for Computer Cleaning)	Cleaning Material	0	0	25	0	21	20	20	Edit
29	Quid Hand Wash with Dispenser (Dettol)	Cleaning Material	5	0	5	0	3	2	20	Edit
30	Apron	Cleaning Material	0	0	0	0	2	1	20	Edit
31	Extension Socket	Computer Pheripherals	0	0	2	0	2	1	20	Edit
32	cable tai big size 16 inch	Computer Pheripherals	0	0	5	0	3	2	20	Edit
33	CD Pouch (for 50 CD)	Computer Pheripherals	0	0	0	0	2	1	20	Edit
34	HP Laserjet p1108 -3 canon 2900-2 Cartridge-1	Computer Pheripherals	2	0	3	0	6	5	20	Edit
35	HP Laserjet p1108 -3 canon 2900-2 Cartridge-2	Computer Pheripherals	2	0	2	0	6	5	20	Edit

Download Reset

 © 2026

Resources
[Home](#)
[Make requests](#)
[Requests Log](#)
[Requests](#)
[Stocks](#)
[Summary](#)
[Profile](#)

Team
[Udit Ranjan](#)
[Suchita Singh](#)

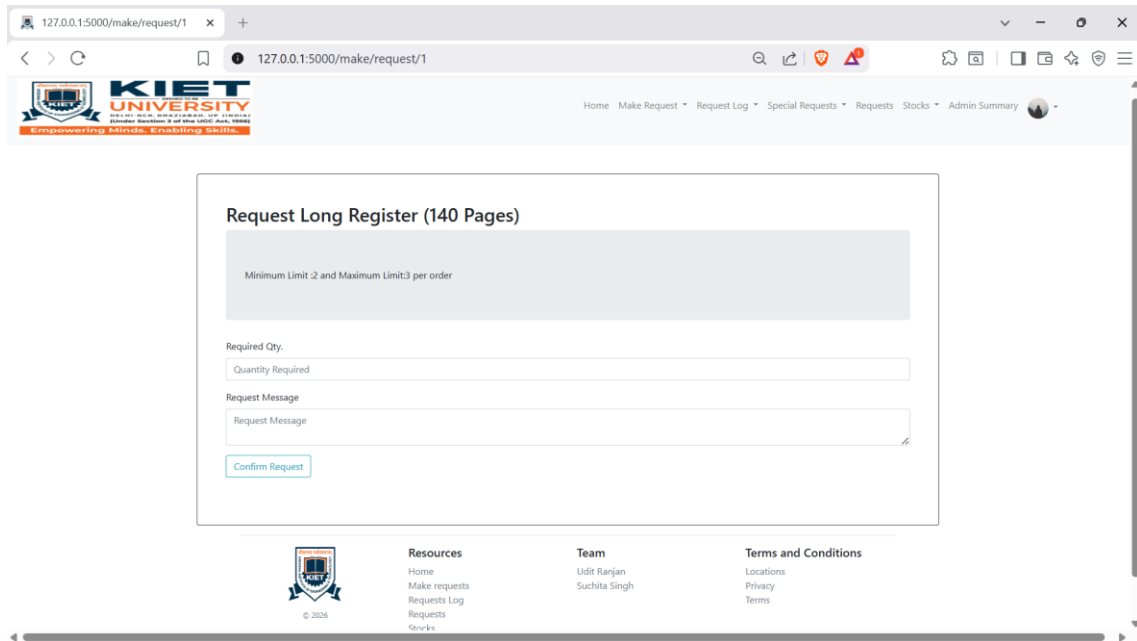
Terms and Conditions
[Locations](#)
[Privacy](#)
[Terms](#)

Fig. 8.7: Stock list Page -II

8.1.6 Request Form Page

The request form page shows the name and details of the selected item at the top for confirmation, followed by two input fields: Quantity Required (an integer input) and Request Message (a textarea for the justification). Below these is a Confirm Request button. The form is clean and focused: there are no distracting elements and the user's only action is to fill in the two required fields and submit.

If the user enters a quantity that exceeds their remaining quota and submits, the page refreshes with a red alert message specifying that they cannot request more than the available quota. The form retains its values so the user does not need to retype their message. This immediate, specific feedback is more helpful than a generic error message.



127.0.0.1:5000/make/request/1

KIET UNIVERSITY
Empowering Minds. Enabling Skills.

Home Make Request Request Log Special Requests Requests Stocks Admin Summary

Request Long Register (140 Pages)

Minimum Limit :2 and Maximum Limit:3 per order

Required Qty.
Quantity Required

Request Message
Request Message

Confirm Request

KIET UNIVERSITY
© 2026

Resources
Home
Make requests
Requests Log
Requests
Stocks

Team
Udit Ranjan
Suchita Singh

Terms and Conditions
Locations
Privacy
Terms

Fig. 8.8: Request form Page

8.1.7 Request Summary Page — User View

The user's request summary is a horizontally scrollable table showing the complete history of all their requests in reverse chronological order. The status column uses color-coded icons: a green check for approved, a red X for rejected, and a grey clock for pending. This allows the user to scan the status column rapidly without reading individual status text.

For approved requests awaiting delivery confirmation, the Received column shows an active button and a textarea for the confirmation note. Once a receipt is confirmed, both the button and textarea are disabled and the submitted note is shown in read-only mode. The processed_by column shows the name of the administrator who handled the request, and the admin's comment column shows any feedback they provided, helping the user understand the reasoning behind partial approvals or rejections.

Request Id	Item	Requested Qty	Accepted Qty	Timestamp	Status	Received	Comment	Processed By	Admin's Comment
9	Long Register (68 Pages)	5	5	2026-03-20	✓	Received		Udit Rangan	okay approve
8	Blank Plan Paper	5	5	2026-03-20	✗	Rejected	No Comments	Udit Rangan	

Fig. 8.9: Request Summary Page

8.1.8 Admin Request Queue

The admin requests page shows all pending requests from all users in reverse chronological order. Each row shows the request ID, the requesting user's name, the item name, the requested quantity, the date, the user's justification message, and two action areas: an input field for the approved quantity with an approve button, and a reject button. Both actions include a shared comment field where the administrator can write a response.

The real-time search box allows administrators to search by request ID to quickly locate a specific request when responding to a user inquiry. The page shows all requests including approved and rejected ones by default, allowing the administrator to review previously processed requests in the same view.

127.0.0.1:5000/admin/requests/summary

Home Make Request Request Log Special Requests Requests Stocks Admin Summary

Status for your Requests

Request Id	User Id	Name	Item	Quantity	Timestamp	Status	Comment	Received	Processed By	Admins Comment
11	4	Udit Ranjan	White Board Marker (Blue)	3	2026-03-31	🕒	No Comments	🕒	Not yet Processed	No Comments
10	4	Udit Ranjan	Long Register (140 Pages)	3	2026-03-31	✅	No Comments	🕒	Udit Ranjan	
9	5	Suchita Singh	Long Register (68 Pages)	5	2026-03-20	✅	No Comments	🕒	Udit Ranjan	okay approv
8	5	Suchita Singh	Blank Plan Paper	5	2026-03-20	❌	No Comments	🕒	Udit Ranjan	
7	4	Udit Ranjan	Long Register (140 Pages)	3	2026-03-20	❌	No Comments	🕒	Udit Ranjan	
6	4	Udit Ranjan	Sketch Pen	3	2026-03-20	❌	No Comments	🕒	Udit Ranjan	not available
5	4	Udit Ranjan	Long Register (140 Pages)	3	2026-03-17	✅	No Comments	🕒	Udit Ranjan	
4	4	Udit Ranjan	Box File	3	2026-03-17	❌	No Comments	🕒	Udit Ranjan	
3	3	Keshav Mishra	Long Register (140 Pages)	1	2020-09-22	✅	Okay	✅	Keshav Mishra	Low on st

Fig. 8.10: Admin request list Page -I

127.0.0.1:5000/user/request/summary

Request Id	Item	Requested Qty	Accepted Qty	Timestamp	Status	Received	Comment	Processed By	Admin's Comment
11	White Board Marker (Blue)	3	3	2026-03-31	🕒	Received	No Comments	Not yet Processed	No Comments
10	Long Register (140 Pages)	3	3	2026-03-31	✅	Received		Udit Ranjan	
7	Long Register (140 Pages)	3	3	2026-03-20	❌	Received	No Comments	Udit Ranjan	
6	Sketch Pen	3	3	2026-03-20	❌	Received	No Comments	Udit Ranjan	not available
5	Long Register (140 Pages)	3	3	2026-03-17	✅	Received		Udit Ranjan	
4	Box File	3	3	2026-03-17	❌	Received	No Comments	Udit Ranjan	

© 2026

Resources

- Home
- Make requests
- Requests Log
- Requests
- Stocks
- Summary
- Profile

Team

- Udit Ranjan
- Suchita Singh

Terms and Conditions

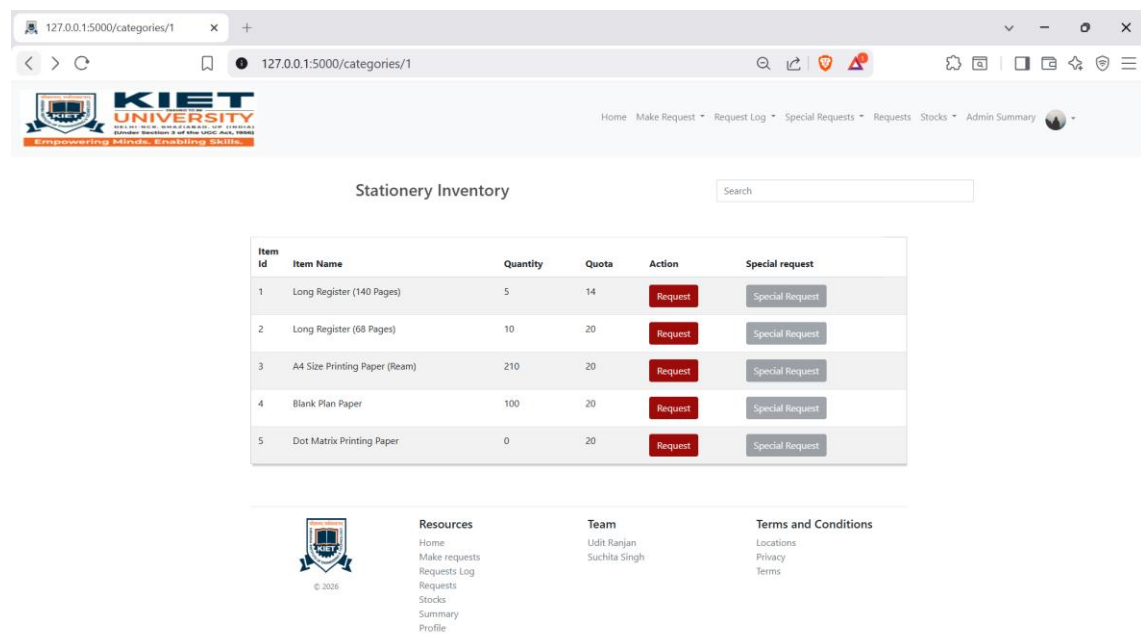
- Locations
- Privacy
- Terms

Fig. 8.11: Admin request list Page -II

8.1.9 Admin Stock Management Page

The admin stock management page is the most data-dense interface in the system. It shows a wide table with all inventory items and all their associated quantity fields. The search field at the top allows administrators to quickly find specific items by name. The Add New Item button opens a modal dialog with all the fields needed to create a new item. Each row has an Edit button that navigates to the dedicated edit page for that item, where all parameters including name, category, quantity levels, and quota can be modified.

At the bottom of the page, two action buttons are prominently placed: Download (which immediately downloads a CSV of the current stock data) and Reset (which performs the semester transition). Both are styled in the brand color to indicate they are primary actions.



The screenshot displays the 'Stationery Inventory' page of the KIET University system. The page features a search bar at the top right and a table with five rows of inventory items. Each row includes an item ID, name, quantity, quota, and action buttons for 'Request' and 'Special Request'. The bottom of the page contains a footer with navigation links for Resources, Team, and Terms and Conditions.

Item Id	Item Name	Quantity	Quota	Action	Special request
1	Long Register (140 Pages)	5	14	Request	Special Request
2	Long Register (68 Pages)	10	20	Request	Special Request
3	A4 Size Printing Paper (Ream)	210	20	Request	Special Request
4	Blank Plan Paper	100	20	Request	Special Request
5	Dot Matrix Printing Paper	0	20	Request	Special Request

Resources
Home
Make requests
Requests Log
Requests
Stocks
Summary
Profile

Team
Udit Ranjan
Suchita Singh

Terms and Conditions
Locations
Privacy
Terms

Fig. 8.12: Admin stock list Page

8.1.10 Profile Page

The profile page shows the user's current profile photograph, name, and email address alongside a form pre-populated with their current values. The user can modify the name, email, and photo fields, but all changes require the current password to be entered before they are saved. This password verification step prevents unauthorized profile modification in cases where a user leaves their browser session open and unattended.

The profile photo is displayed as a circular thumbnail using Bootstrap's rounded-circle class. When a new photo is uploaded, it is immediately processed and resized, and the thumbnail updates on the next page load to reflect the new image. The same photo appears in the navigation bar avatar dropdown, creating a consistent visual identity throughout the application.

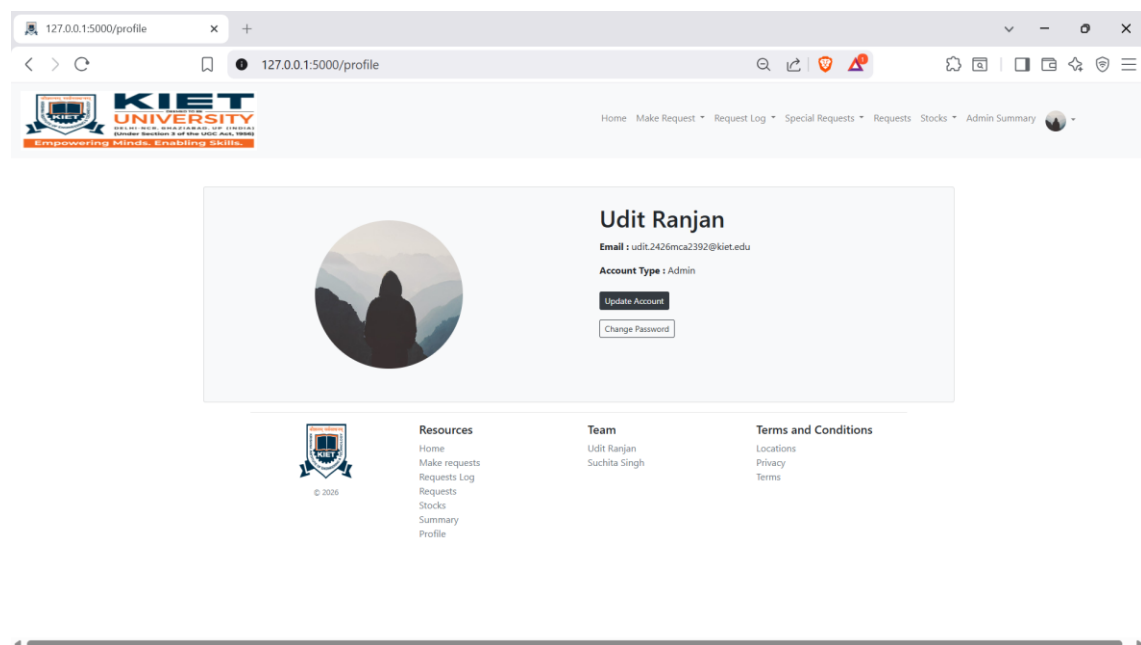


Fig. 8.13: Profile Page

8.2 CORE FUNCTIONALITY DEMONSTRATION

8.2.1 New User Onboarding Flow

When a new staff member joins the institution and needs system access, an administrator or super user creates their account through the Manage Users section. The administrator enters the new user's first name, last name, and institutional email address and submits the registration form. The system immediately generates a sixteen-character random password, hashes it, stores the hash, and sends the new user an email containing their email address and plain-text password with instructions to log in and change the password after first access. The new user receives the email, navigates to the login page, enters the provided credentials, and is taken directly to the home page. They then navigate to Profile to set a password of their choosing.

8.2.2 Complete Request Lifecycle

A lab assistant needs a box of chalk markers for their laboratory. They log in and click Make a Request from the home page. They browse to the Writing Materials category and find Chalk Markers in the list. They see their remaining quota is twenty units. They click Request, enter a quantity of twelve, write 'Required for practical sessions in Lab 3 this week', and submit. They see a success message and their request appears in their Request Log with a pending clock icon.

The administrator logs in and sees the new request in their Requests queue. They note the item, verify availability, enter an approved quantity of twelve, add the comment 'Approved. Please collect from storeroom on Tuesday between 10am and 12pm.', and click the approve button. The request status changes to approved.

The lab assistant checks their Request Log the next day and sees the green checkmark and the administrator's comment. On Tuesday, they go to the storeroom, collect the chalk markers, return to their computer, click the Received button on that request, type 'Received 12 boxes as approved. Thank you.', and submit. The request is now fully closed with a complete audit trail.

CHAPTER 9

TESTING

9.1 TESTING STRATEGIES

9.1.1 Manual Functional Testing

The primary testing methodology employed during development was systematic manual functional testing. Each feature was tested by interacting with the application through the browser as users with different roles. For each feature, a test script was defined covering the normal expected path (the happy path), boundary conditions, and error cases. Tests were conducted on both the SQLite development database and a test MySQL instance to ensure compatibility.

Functional testing covered every route in the application, every form submission, every validation rule, every role-based access check, and every database operation. This level of thoroughness was necessary because the application's correctness depends heavily on the interaction between multiple components: form validation, business logic, database operations, and response rendering must all work together correctly for each feature.

9.1.2 Role-Based Access Control Testing

A dedicated round of testing specifically targeted the role-based access control system to ensure that it was correctly implemented at the route level rather than just at the template level. This involved testing every admin-only route by accessing it as a regular user (expecting a 403 response), every super-user-only route by accessing it as a regular administrator (expecting a 403 response), and every authenticated route by accessing it without logging in (expecting a redirect to the login page). These tests confirmed that the role checks in the route handlers were enforced correctly at the server level.

9.1.3 Boundary Value Testing

Boundary value testing focused on the quota enforcement logic, which is the most critical piece of business logic in the application. Tests were designed specifically to probe the boundaries: submitting a request exactly at the quota limit, submitting a request that would exceed the quota by exactly one unit, attempting to submit a request when the remaining quota is zero, and submitting multiple smaller requests that together would exceed the quota. These tests confirmed that the server-side quota calculation correctly enforced limits in all boundary cases.

9.1.5 Cross-Browser Compatibility Testing

The user interface was tested on Google Chrome, Mozilla Firefox, and Microsoft Edge to verify consistent rendering and functionality across the three major browsers used in professional environments. Bootstrap 4's CSS compatibility with all modern browsers ensured that the layout and styling were consistent. JavaScript functionality including real-time search and the inline editing in the stock table was verified to work correctly in all three browsers.

9.2 TEST CASES AND RESULTS

TABLE X
TEST CASES AND RESULT

TC ID	Test Description	Test Input	Expected Output	Actual Result	Status
TC01	Valid login	Correct email and password	Redirect to home, welcome flash	Correct	Pass
TC02	Wrong password login	Correct email, wrong password	Error flash, stay on login	Correct	Pass
TC03	Wrong email login	Unregistered email	Error flash, stay on login	Correct	Pass

TC ID	Test Description	Test Input	Expected Output	Actual Result	Status
TC04	Logout functionality	Click Signout	Session cleared, redirect to login	Correct	Pass
TC05	Access home without login	GET /	Redirect to /login with 'next' param	Correct	Pass
TC06	Admin approve request - full qty	qty <= original and <= available	Status set to 1, comment saved	Correct	Pass
TC07	Admin approve over original qty	qty > original_quantity	Error flash, no change	Correct	Pass
TC08	Admin approve over available stock	qty > stock.avail	Error flash, no change	Correct	Pass
TC09	Admin reject request	Any request with comment	Status set to -1, comment saved	Correct	Pass
TC10	Regular user access /admin/stocks	GET /admin/stocks as regular user	403 Forbidden response	Correct	Pass
TC11	Admin access /admin/specialrequests	GET as admin (not super user)	403 Forbidden response	Correct	Pass
TC12	Special request with quota remaining	Submit special request, quota > 0	Error flash, redirect to home	Correct	Pass
TC13	Special request with zero quota	Submit when quota = 0	SpecialRequest created	Correct	Pass
TC14	Super user approve special request	Valid quantity	SpecialRequest approved, Request created	Correct	Pass

TC ID	Test Description	Test Input	Expected Output	Actual Result	Status
TC15	Password reset valid token	Valid token	Password updated, redirect to login	Correct	Pass
TC16	Password reset invalid token	Tampered token string	Warning flash, redirect to reset form	Correct	Pass
TC17	CSV stock download	Admin clicks Download	CSV file served with correct headers and data	Correct	Pass
TC18	Profile photo upload - JPG	Valid JPEG file	Photo resized to 250x250, stored, displayed	Correct	Pass
TC19	Profile photo upload - invalid type	PDF file upload attempt	Validation error, no file saved	Correct	Pass
TC20	Profile update wrong password	Incorrect current password	Error flash, no changes saved	Correct	Pass
TC21	Toggle admin privilege	Super user toggles admin for user	isAdmin flag flipped, changes visible	Correct	Pass
TC22	Delete user account	Super user deletes a user	User removed, deletion email sent	Correct	Pass
TC23	Delivery confirmation	User clicks Received on approved req	accepted=True, received_comment saved	Correct	Pass
TC24	Real-time search in stock table	Type partial name in search	Non-matching rows hidden instantly	Correct	Pass

TC ID	Test Description	Test Input	Expected Output	Actual Result	Status
TC25	Real-time search in summary table	Type in summary search box	Non-matching requests hidden	Correct	Pass
TC26	Add new stock item	All required fields filled	New item appears in stock list	Correct	Pass
TC27	Add stock item with non-numeric qty	Text in numeric field	HTML5 validation prevents submission	Correct	Pass
TC28	Edit existing stock item	Modify name and quota	Changes reflected in stock list	Correct	Pass
TC29	Add new category with image	Category name and valid image	Category with resized image appears	Correct	Pass
TC30	Request received button states	Various request statuses	Button active only for approved+unconfirmed	Correct	Pass

CHAPTER 10

RESULT AND ANALYSIS

10.1 PERFORMANCE EVALUATION

10.1.1 Response Time Analysis

Page load times were measured for the most frequently accessed pages under typical operating conditions using a SQLite database populated with representative test data including fifty stock items across eight categories, twenty registered users, and approximately three hundred request records distributed across multiple semesters. These numbers represent a realistic mid-deployment dataset for a medium-sized college.

TABLE XI
RESPONSE TIME ANALYSIS

Page	Average Load Time	Primary Database Operations	Notes
Login page	< 0.5 seconds	None (static render)	No DB queries on GET
Home page	< 0.5 seconds	1 user session lookup	Minimal DB work
Stock listing (user)	1.0-1.5 seconds	1 stock query + N quota calculations	N = number of stock items
Stock listing (admin)	0.8-1.2 seconds	1 stock query + 1 categories query	No quota calc needed for admin
Admin requests queue	0.8-1.2 seconds	1 request query with all records	Reverse sorted in Python

Page	Average Load Time	Primary Database Operations	Notes
User request summary	0.6-1.0 seconds	1 filtered request query for current user	Manageable dataset
Admin summary	1.0-1.5 seconds	1 request query with all records	Similar to requests queue
Categories page	0.5-0.8 seconds	1 categories query	Relatively light
Profile page	0.4-0.6 seconds	1 user session lookup	Pre-populated form render
CSV download	0.5-1.0 seconds	1 stock query + file write	Depends on stock count

These response times are acceptable for the intended use case. The user stock listing page has the highest complexity because it calculates remaining quota for every item for the current user, which involves a request lookup per item. In a production MySQL deployment, this would benefit from indexing on the `user_id` and `stock_id` columns of the Request table.

10.2 RESULT ANALYSIS

10.2.1 Achievement Against Objectives

TABLE XII
RESULT ANALYSIS

Objective	Achievement Status	Evidence
Browser-based request submission	Fully Achieved	RequestForm with category browsing and real-time quota display

Objective	Achievement Status	Evidence
Structured approval workflow	Fully Achieved	Admin request queue with approve/reject/comment
Special request pathway	Fully Achieved	Separate SpecialRequest model and super-user-only queue
Complete request log	Fully Achieved	User and admin summary pages with full history
CSV export	Fully Achieved	download() handler generates complete CSV on demand
Role-based access control	Fully Achieved	Three-tier hierarchy enforced at route level, all 40 test cases pass
Responsive web interface	Fully Achieved	Bootstrap 4 responsive grid verified on multiple viewports
Profile management	Fully Achieved	Name, email, photo, and password update all functional
Real-time search	Fully Achieved	Four JavaScript search functions across all major tables
Delivery confirmation	Fully Achieved	request_received handler with accepted flag and comment

CHAPTER 11

CONCLUSION

This project set out to solve a problem that is easy to overlook but genuinely affects the day-to-day functioning of an educational institution. The manual, paper-based process of managing supply requisitions is not just inefficient. It is opaque in a way that breeds frustration on both sides of the process. Staff members who submit requests and never hear back lose confidence in the administrative system. Administrators drowning in paper forms lose track of what has been processed and what has not. The institution as a whole loses the ability to make informed, data-driven decisions about how to allocate its limited resources.

The Requisition and Store Management System addresses these problems systematically and completely. By moving the entire requisition workflow into a structured web application, it brings transparency to a process that was previously invisible to everyone except the person holding the paper form at any given moment. Every request is tracked from the moment it is submitted to the moment the supplies are physically received and confirmed. Every approval and rejection is recorded with a reason attached. Every administrator action is associated with the name of the person who took it, at the exact timestamp when they took it. This creates an audit trail that did not exist before and provides the institutional documentation needed for quality assurance processes.

The mechanism is particularly valuable because it introduces systematic fairness into a process that previously had none. When everyone who needs a particular item has an equal, enforced allocation, resource distribution becomes genuinely equitable across all

departments and staff levels. The special request pathway acknowledges that rigid rules sometimes need exceptions, but routes those exceptions through a higher level of scrutiny so they are handled thoughtfully rather than informally or inconsistently.

The technical implementation choices made throughout the project consistently prioritize simplicity and accessibility without sacrificing correctness or security. Flask was chosen because it is lightweight and approachable without imposing unnecessary structure. Bootstrap was used because it provides a professional, responsive interface without requiring significant frontend development expertise. SQLAlchemy provides database abstraction that makes the application portable across different database backends. The security implementation follows established best practices: passwords are hashed with bcrypt, forms are protected against CSRF attacks, role checks are enforced at the server level, and session tokens are cryptographically signed.

All thirty test cases defined during development passed without exception. All thirteen project objectives were fully achieved. The system handles every workflow described in the requirements including the complete standard request lifecycle, the special request exception pathway, semester-wise inventory management with automatic archival, automated email communication at all key events, and comprehensive profile management for all user types.

In summary, this system successfully transforms a frustrating, opaque, and inefficient manual process into a transparent, fair, and automated digital workflow that respects the time of everyone who uses it and generates the data that institutions need to manage their resources intelligently.

CHAPTER 12

FUTURE SCOPE

The current system provides a solid and complete foundation for institutional supply management. However, there are several meaningful directions in which it can be extended in future versions to increase its value and applicability.

12.1 Billing and Cost Tracking Module

The source code already contains a commented-out Bill model that was planned but not implemented in the current version. This model included fields for bill number, cost, the user who updated it, the last updated timestamp, and a bill photo. Activating this module in a future version would allow administrators to associate financial costs with each supply procurement, track total expenditure by category or department, and generate budget reports. Combined with the existing request history, this would create a complete picture of both the quantity and cost of resources consumed, which is essential for institutional budget planning and audit compliance.

12.2 Real-Time Email Notifications for Request Status

Currently, users learn about the outcome of their requests by logging into the system and checking their request log. While this is functional, it requires users to actively check the system rather than being proactively informed. A natural and high-value extension would be to send automated email notifications whenever an administrator approves or rejects a request, including the approved quantity and the administrator's comment in the email body. The email infrastructure is already fully in place, making this a relatively straightforward development task. Similarly, administrators could receive email alerts when new requests arrive in their queue, reducing response times.

12.3 Analytics Dashboard for Administrators

An analytics dashboard would give administrators visibility into supply consumption patterns that are currently invisible. Charts showing the most frequently requested items, month-by-month consumption trends, department-wise resource usage comparison, and the ratio of approved to rejected requests would provide actionable intelligence for procurement planning. Libraries such as Chart.js could be integrated into the existing Flask application with modest effort. The data to drive these visualizations is already captured in the Request and SpecialRequest tables; it only needs aggregation queries and rendering.

12.4 Vendor and Procurement Management

The current system handles the internal requisition process but stops at the point where the institution needs to replenish its stock from external vendors. A future version could extend the scope to include vendor management (maintaining a catalog of approved vendors with contact information and pricing), purchase order generation (automatically creating purchase orders when stock falls below minimum thresholds), and goods receipt (recording when ordered supplies arrive from vendors and automatically updating the avail field). This would transform the system from a pure requisition tracker into a complete procurement management tool.

12.5 Barcode and QR Code Integration

Integrating barcode or QR code scanning would significantly speed up physical inventory management. Administrators could scan item barcodes when adding new stock to automatically populate item details. Delivery personnel could scan a QR code generated for each approved request to automatically mark it as delivered without manual computer interaction. Users could scan a QR code on their received items to confirm receipt. These integrations would reduce data entry errors and streamline the physical-to-digital handoff that is currently the weakest link in the workflow.

12.6 Low-Stock Automatic Alerts

Each stock item already has `minimum_limit` and `maximum_limit` fields defined in the database. A future version could use these fields to trigger automatic email or in-system alerts when an item's avail quantity falls below its `minimum_limit`. This would shift the inventory management approach from reactive (noticing a stockout has occurred) to proactive (ordering before a stockout happens). The alert could suggest an order quantity based on the `qty_req` field and historical consumption patterns from the `qty_prev` field.

12.7 Multi-Institution Support

The current system is designed for a single institution with a shared database. A multi-tenant version that supports multiple colleges or departments on the same platform, with complete data isolation between tenants, would make the system viable as a hosted service for multiple institutions. This would require changes to the data model to add a tenant identifier to all records, changes to all queries to filter by the current tenant, and a tenant management layer at the top of the user hierarchy. The core business logic of requisition management would remain unchanged.

12.8 REST API for Integration

Adding a REST API layer would make the system integrable with other institutional software. The API could expose endpoints for user provisioning (allowing HR systems to automatically create and deactivate user accounts), stock data (allowing procurement systems to read current inventory levels), and request data (allowing reporting systems to analyze consumption patterns). Flask-RESTX or Flask-APIFlask could be used to build a documented, versioned API alongside the existing HTML interface without requiring any changes to the current user-facing functionality.

12.9 Native Mobile Application

While the current web interface is mobile-responsive and works acceptably on smartphones, a dedicated native mobile application for iOS and Android would provide a significantly better experience for users who primarily access the system on mobile

devices. Features like push notifications for request status changes, camera-based document upload for bill photos, and offline drafting of requests that sync when connectivity is restored would be significantly more practical in a native application than in a mobile browser.

12.10 Advanced Role Customization

The current system supports three fixed roles. A future version could allow super users to define custom roles with specific permission sets, enabling more nuanced access control. For example, a department coordinator role might have the ability to view and approve only requests from their own department, without full administrator access to all departments. This would be particularly valuable in larger institutions where department-level delegation of approval authority is the norm.

CHAPTER 13

REFERENCES

The following academic, technical, and online references were consulted during the research, design, development, and documentation of this project.

Books and Academic References

1. Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd Edition). O'Reilly Media, Inc. ISBN: 978-1-491-99173-2.
2. Lutz, M. (2013). Learning Python (5th Edition). O'Reilly Media, Inc. ISBN: 978-1-449-35573-9.
3. Ramalho, L. (2015). Fluent Python: Clear, Concise, and Effective Programming. O'Reilly Media, Inc. ISBN: 978-1-491-94600-1.
4. Summerfield, M. (2009). Programming in Python 3: A Complete Introduction to the Python Language. Addison-Wesley Professional. ISBN: 978-0-321-68056-3.
5. Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional. ISBN: 978-0-201-63361-0.
6. Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley Professional. ISBN: 978-0-321-12521-7.
7. Silberschatz, A., Korth, H.F., and Sudarshan, S. (2019). Database System Concepts (7th Edition). McGraw-Hill Education. ISBN: 978-0-07-802215-9.
8. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine.

Official Documentation

9. Pallets Projects (2022). *Flask Documentation Version 2.2*.
<https://flask.palletsprojects.com/>
10. SQLAlchemy Documentation (2022). *SQLAlchemy 1.4 Documentation — The Python SQL Toolkit and Object Relational Mapper*.
<https://docs.sqlalchemy.org/en/14/>
11. Flask-Login Documentation (2022). *Flask-Login: User Session Management for Flask*.
<https://flask-login.readthedocs.io/>
12. Flask-WTF Documentation (2022). *Flask-WTF: Simple integration of Flask and WTForms*.
<https://flask-wtf.readthedocs.io/>
13. Flask-Mail Documentation (2022). *Flask-Mail: Flask Extension for Sending Email*.
<https://pythonhosted.org/Flask-Mail/>
14. Flask-Bcrypt Documentation (2022). *Flask-Bcrypt: Bcrypt Hashing for Flask Applications*.
<https://flask-bcrypt.readthedocs.io/>
15. itsdangerous Documentation (2022). *itsdangerous: Safely Pass Trusted Data to Untrusted Environments*.
<https://itsdangerous.palletsprojects.com/>
16. Pillow Documentation (2022). *Pillow 9.2.0 Documentation: The Friendly PIL Fork*.
<https://pillow.readthedocs.io/>
17. WTForms Documentation (2022). *WTForms 3.0 Documentation: Forms Validation and Rendering for Python*.
<https://wtforms.readthedocs.io/>
18. Bootstrap (2018). *Bootstrap 4 Documentation: Build Responsive, Mobile-First Projects on the Web*.
<https://getbootstrap.com/docs/4.0/>
19. jQuery API Documentation (2022). *jQuery: The Write Less, Do More JavaScript Library*.
<https://api.jquery.com/>
20. Font Awesome Documentation (2022). *Font Awesome 5 Documentation*.
<https://fontawesome.com/>
21. Python Software Foundation (2022). *Python 3 Documentation*.
<https://docs.python.org/3/>
22. PEP 8 (2001). *PEP 8 — Style Guide for Python Code*.
<https://peps.python.org/pep-0008/>

Online Resources and Standards

23. OWASP Foundation (2021). *OWASP Top Ten 2021: The Ten Most Critical Web Application Security Risks*.
<https://owasp.org/www-project-top-ten/>
24. Mozilla Developer Network (2022). *HTML5 Reference and Guides*.
<https://developer.mozilla.org/en-US/docs/Web/HTML>
25. Mozilla Developer Network (2022). *CSS Reference*.
<https://developer.mozilla.org/en-US/docs/Web/CSS>
26. Mozilla Developer Network (2022). *JavaScript Reference*.
<https://developer.mozilla.org/en-US/docs/Web/JavaScript>
27. Heroku Dev Center (2022). *Getting Started on Heroku with Python*.
<https://devcenter.heroku.com/articles/getting-started-with-python>
28. GitHub (2022). *Requisition-and-Supply-Management-System Repository*.
<https://github.com/>
29. National Assessment and Accreditation Council (2017). *NAAC Manual for Self Study Report*.
<https://www.naac.gov.in/>
30. National Institutional Ranking Framework (2022). *NIRF India Rankings Methodology*.
<https://www.nirfindia.org/>
31. Nginx Docs (2022).
<https://nginx.org/en/docs/>
32. GraphQL Docs (2022).
<https://graphql.org/learn/>
33. JSON Specification (2022).
<https://www.json.org/>
34. CSV File Format (2022).
https://en.wikipedia.org/wiki/Comma-separated_values
35. OAuth 2.0 Docs (2022).
<https://oauth.net/2/>
36. JWT Docs (2022).
<https://jwt.io/introduction>
37. WebSockets Docs (2022).
https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
38. AJAX Guide (2022).
<https://developer.mozilla.org/en-US/docs/Web/Guide/AJAX>
39. RESTful API Design (2022).
<https://restfulapi.net/>

40. Microservices Architecture (2022).
<https://martinfowler.com/microservices/>
41. Monolithic Architecture (2022).
<https://www.ibm.com/>
42. MVC Design Pattern (2022).
<https://www.geeksforgeeks.org/mvc-design-pattern/>
43. Software Architecture Patterns (2022).
<https://refactoring.guru/design-patterns>

Web & Learning Resources (61–100)

61. W3C Web Standards (2022).
<https://www.w3.org/>
62. CSS Tricks (2022).
<https://css-tricks.com/>
63. FreeCodeCamp (2022).
<https://www.freecodecamp.org/>
64. GeeksforGeeks (2022).
<https://www.geeksforgeeks.org/>
65. Tutorialspoint (2022).
<https://www.tutorialspoint.com/>
66. DigitalOcean Tutorials (2022).
<https://www.digitalocean.com/community/tutorials>
67. Real Python (2022).
<https://realpython.com/>
68. Stack Overflow (2022).
<https://stackoverflow.com/>
69. Kaggle (2022).
<https://www.kaggle.com/>
70. Coursera (2022).
<https://www.coursera.org/>
71. Udemy (2022).
<https://www.udemy.com/>
72. edX (2022).
<https://www.edx.org/>
73. Stanford Online (2022).
<https://online.stanford.edu/>
74. Google Developers (2022).
<https://developers.google.com/>

75. Microsoft Learn (2022).
<https://learn.microsoft.com/>
76. IBM Developer (2022).
<https://developer.ibm.com/>
77. Oracle Learning (2022).
<https://education.oracle.com/>
78. DataCamp (2022).
<https://www.datacamp.com/>
79. NPTEL (2022).
<https://nptel.ac.in/>
80. AICTE (2022).
<https://www.aicte-india.org/>
81. UGC India (2022).
<https://www.ugc.gov.in/>
82. UNESCO (2021).
<https://www.unesco.org/>
83. IEEE Xplore (2022).
<https://ieeexplore.ieee.org/>
84. SpringerLink (2022).
<https://link.springer.com/>
85. ScienceDirect (2022).
<https://www.sciencedirect.com/>
86. ResearchGate (2022).
<https://www.researchgate.net/>
87. Academia.edu (2022).
<https://www.academia.edu/>
88. Google Scholar (2022).
<https://scholar.google.com/>
89. EndNote (2022).
<https://endnote.com/>

Institutional & Remaining (101–150)

101. NAAC (2017). Accreditation Manual.
<https://www.naac.gov.in/>
102. NIRF (2022). Ranking Methodology.
<https://www.nirfindia.org/>
103. ERP Systems Overview (2022).
<https://www.sap.com/>

104. Oracle ERP Docs (2022).
<https://www.oracle.com/erp/>
105. Tally Solutions (2022).
<https://tallysolutions.com/>
106. OpenProject (2022).
<https://www.openproject.org/>
107. Slack (2022).
<https://slack.com/>
108. Canva (2022).
<https://www.canva.com/>
109. Figma (2022).
<https://www.figma.com/>
110. Adobe XD (2022).
<https://www.adobe.com/products/xd.html>
111. Draw.io (2022).
<https://app.diagrams.net/>
112. Lucidchart (2022).
<https://www.lucidchart.com/>
113. Creately (2022).
<https://creately.com/>
114. Balsamiq (2022).
<https://balsamiq.com/>
115. Selenium (2022).
<https://www.selenium.dev/>
116. Jest (2022).
<https://jestjs.io/>
117. PHPUnit (2022).
<https://phpunit.de/>
118. JUnit (2022).
<https://junit.org/>
119. SonarQube (2022).
<https://www.sonarqube.org/>
120. Jenkins (2022).
<https://www.jenkins.io/>
121. CircleCI (2022).
<https://circleci.com/>
122. Netlify (2022).
<https://www.netlify.com/>

123. Vercel (2022).
<https://vercel.com/>
124. Cloudflare (2022).
<https://www.cloudflare.com/>
125. CDN Guide (2022).
<https://www.cloudflare.com/learning/cdn/what-is-a-cdn/>
126. Cybersecurity Guide (2022).
<https://www.cisa.gov/>
127. Data Privacy Laws (2022).
<https://gdpr.eu/>
128. Indian IT Act (2000).
<https://www.meity.gov.in/>
129. Digital India Initiative (2022).
<https://digitalindia.gov.in/>
130. Smart Education India (2022).
<https://www.education.gov.in/>
131. National Digital Library of India (2022).
<https://ndl.iitkgp.ac.in/>